BreakBite

Smart Food, Smart Time

প্রজেক্টের সম্পূর্ণ ডকুমেন্টেশন ও পুনর্নির্মাণ গাইড (বাংলা সংস্করণ)

Team BreakByte

Gono Bishwabidyalay - CSE Software Development Contest 2026

Category 1: Semester 1st to 4th

Document date: 20 October 2026

Live site: https://raito-del.github.io/BreakBite/

Repository: github.com/raito-del/BreakBite

গোপনীয় টিম ডকুমেন্ট। এতে কোনো পাসওয়ার্ড বা প্রাইভেট কী নেই।

সূচিপত্র

1. টিম ও দায়িত্ব

2. প্রজেক্টের ধারণা

3. চিন্তাভাবনা ও ডিজাইনের সিদ্ধান্ত

4. ফিচারের পূর্ণ তালিকা

5. আর্কিটেকচার ও প্রযুক্তি

6. ফাইল তালিকা (প্রতিটা ফাইল)

7. ডেটা মডেল (Firestore)

8. মূল ফিচারগুলো কীভাবে কাজ করে

9. ডিজাইন সিস্টেম ও থিম

10. নিরাপত্তার অবস্থা ও জানা সীমাবদ্ধতা

11. পুনর্নির্মাণ গাইড (ধাপে ধাপে)

12. কাজের লগ ও টাইমলাইন

13. ভবিষ্যতের কাজ

আপডেট ১. অ্যানিমেটেড লোগো ও স্প্ল্যাশ পেজ

শব্দের মানে (গ্লসারি)

পরিশিষ্ট (কোড, ইংরেজিতে): Appendix A. Key source code এবং Appendix B. Complete source of every file

নোট: ফাইলের নাম, ফিল্ডের নাম আর প্রযুক্তির নাম ইংরেজিতেই রাখা হয়েছে। কোডগুলো ইংরেজিতেই আছে এবং এই ডকুমেন্টের শেষে Appendix A ও Appendix B-তে দেওয়া হয়েছে।

1. টিম ও দায়িত্ব

টিমের নাম: BreakByte। প্রজেক্টের নাম: BreakBite। আমরা একটাই সংযুক্ত ক্যান্টিন ওয়েব অ্যাপ বানাচ্ছি। প্রতিটা সদস্য নির্দিষ্ট কিছু ফাইলের দায়িত্বে আছে, যাতে GitHub-এ কাজ একে অপরের সাথে ধাক্কা না খায়।

নাম	স্টুডেন্ট আইডি	দায়িত্ব	যে ফাইলগুলোর দায়িত্বে
Nahar Mim (Team Lead)	26251037	টিম লিড, ডকুমেন্টেশন, হোম পেজ	index.html, README.md
Sadnan Ahmed	26251038	শিক্ষার্থীদের অর্ডার পেজ	student.html (অর্ডার ফ্লো)
Wasika Jahan	26251026	UI ডিজাইন ও স্টাইলিং	style.css, ভিজ্যুয়াল ডিজাইন
Mirza Muin Labbaik	26251021	স্টাফ পেজ ও ডাটাবেস সেটআপ	staff.html, firebase.js

নাম	স্টুডেন্ট আইডি	দায়িত্ব	যে ফাইলগুলোর দায়িত্বে
Habiba Jannat Taiyaba	26251011	Break Planner ও টেস্টিং	planner.html

শেয়ার্ড ফাইল (style.css, firebase.js) খুব সাবধানে বদলাতে হয়: একজন এডিট করবে, বাকিরা আগে pull করে নেবে। টিম GitHub Desktop ব্যবহার করে। কাজের ধারা: Fetch origin, Pull origin, কাজ, Commit, Push।

2. প্রজেক্টের ধারণা

ক্লাসের মাঝের বিরতি সাধারণত ১৫ থেকে ৪৫ মিনিট হয়, আর শিক্ষার্থীদের হাতে টাকাও কম থাকে। ক্যান্টিনে লম্বা লাইন থাকে, তাই বিরতির একটা অংশ লাইনে দাঁড়িয়েই চলে যায়, আর অনেক সময় কী খাবে সেটা ঠিক করার সময়ও পাওয়া যায় না।

BreakBite এই সমস্যা তিনটা আইডিয়া দিয়ে সমাধান করে:

- **Pre-order (আগে থেকে অর্ডার):** লাইভ মেনু থেকে খাবার বেছে নাও, একটা টোকেন পাও, আর খাবার রেডি হলে নিয়ে যাও।
- **Rush radar:** ক্যান্টিনে এখন কত ভিড় সেটা দেখায়, যাতে কখন যেতে হবে বোঝা যায়।
- **Break Planner:** বিরতির সময় আর টাকার পরিমাণ দিলে সেই সময় ও টাকায় কী কী খাওয়া যায় তার একটা সাজেশন পাওয়া যায়।

স্টাফের দিকটা একটা পেজ থেকেই চলে: লাইভ অর্ডার বোর্ড, মেনু ও দাম নিয়ন্ত্রণ, Food Waste ডিসকাউন্ট, Planner সেটিংস আর দিনশেষের রিপোর্ট।

টার্গেট ইউজার: প্রথম বর্ষের একজন শিক্ষার্থী, যার হাতে ৩০ মিনিট আর কিছু টাকা আছে, এবং যে দ্রুত ও পরিষ্কারভাবে সিদ্ধান্ত নিতে চায়।

3. চিন্তাভাবনা ও ডিজাইনের সিদ্ধান্ত

3.1 কেন সাধারণ HTML, CSS আর JavaScript?

টিমটা এখনো শুরুর দিকের ইউনিভার্সিটি লেভেলে। বিল্ড টুল বা ফ্রেমওয়ার্ক মানে বাড়তি সেটআপ, যেটা টিমমেটদের কম্পিউটারে ভেঙে যেতে পারে। সাধারণ ফাইল Live Server বা GitHub Pages-এ সরাসরি খোলে, আর প্রত্যেক সদস্য কোডটা পড়তে পারে। Firebase CDN মডিউলের মাধ্যমে যোগ করা হয়েছে, তাই npm-এর কোনো ধাপ নেই।

3.2 কেন Firebase?

অর্ডার অনেকগুলো ডিভাইসে রিয়েল টাইমে শেয়ার হতে হয়: শিক্ষার্থীর ফোন আর স্টাফের ট্যাবলেট। Firestore ফ্রি Spark প্ল্যানে লাইভ আপডেট (onSnapshot) দেয়। Firebase Authentication স্টাফকে আসল লগইন দেয়, তাই কোডে পাসওয়ার্ড রাখতে হয় না।

3.3 কেন একটাই stylesheet?

টিমের সদস্যরা আলাদা আলাদা থিম চেয়েছিল। সব রং আর আকার একটা style.css ফাইলে CSS variable হিসেবে আছে। মাত্র চারটা রং বদলালে সব পেজের থিম বদলে যায়, কোনো HTML এডিট করতে হয় না।

3.4 কেন শেয়ার্ড অর্ডার কাউন্টার?

দুজন শিক্ষার্থী একই মুহূর্তে বাটন চাপলেও টোকেন নম্বর আলাদা হতে হবে। Firestore transaction counters/orders পড়ে, ১ যোগ করে আর নতুন অর্ডার লেখে, সবই একটাই নিরাপদ ধাপে।

3.5 কেন দৈনিক রিপোর্ট ও রিসেট?

ক্যান্টিন একদিন একদিন করে চলে। সব অর্ডার চিরকাল রেখে দিলে বোর্ড এলোমেলো হয়ে যায় আর টোকেন নম্বর বাড়তেই থাকে। দিনশেষে আগে সারাংশ সেভ হয়, তারপর অর্ডারগুলো মুছে যায় আর টোকেন আবার ১ থেকে শুরু হয়। মুছে ফেলার আগেই রিপোর্ট সেভ হয়, তাই কোনো ধাপ ফেল করলেও বিক্রির তথ্য হারায় না।

3.6 কেন knapsack-স্টাইলের planner?

Planner ছোট আকারের 0/1 আর unbounded knapsack সমস্যা সমাধান করে: প্রতিটা খাবারের একটা সময় আর একটা দাম আছে, আর বাজেটের সময় ও টাকা দুটোরই সীমা আছে। ছোট মেনুর জন্য dynamic programming টেবিল সার্ভার ছাড়াই দ্রুত সেরা উত্তর বের করে দেয়।

3.7 যে ট্রেড-অফগুলো আমরা মেনে নিয়েছি (সৎ দৃষ্টিতে)

- Firestore rules কনটেস্ট ডেমোর জন্য খোলা। আসল টাকা বা ব্যক্তিগত তথ্যের জন্য এটা নিরাপদ না (সেকশন 10 দেখো)।
- Planner আইটেমের সংখ্যা সবচেয়ে বেশি করতে চায়, তাই একটা সস্তা আইটেমই বহুবার দিয়ে ফেলতে পারে। ভ্যারাইটি রুল পরিকল্পনায় আছে।
- Token lookup-এ যে কেউ টোকেন নম্বর দিয়ে অর্ডার দেখতে পারে। তথ্যটা কম সংবেদনশীল, তবে এটা পাবলিক।
- সাইটে বিল্ড স্টেপ নেই, তাই কোড সহজ, কিন্তু মডিউলে ভাগ করা নেই।

4. ফিচারের পূর্ণ তালিকা

এলাকা	ফিচার	কোথায়	অবস্থা
শিক্ষার্থী	দাম, ডিসকাউন্ট ও পাওয়া যাওয়া/না যাওয়াসহ লাইভ মেনু	student.html	সম্পন্ন
শিক্ষার্থী	+ / - দিয়ে পরিমাণ ঠিক করা আর স্টিকি কার্ট বার	student.html	সম্পন্ন
শিক্ষার্থী	transaction-এর মাধ্যমে টোকেনসহ প্রি-অর্ডার	student.html	সম্পন্ন
শিক্ষার্থী	প্রতি ব্রাউজারে একাধিক চলমান অর্ডার সংরক্ষণ	student.html	সম্পন্ন
শিক্ষার্থী	টোকেনের অবস্থার ধাপ (Received, Preparing, Ready)	student.html	সম্পন্ন
শিক্ষার্থী	নম্বর দিয়ে টোকেন খোঁজা	student.html	সম্পন্ন
শিক্ষার্থী	Rush radar (কয়টা অর্ডার অপেক্ষায়)	student.html	সম্পন্ন
শিক্ষার্থী	সময় ও টাকা দিয়ে Break Planner	planner.html	সম্পন্ন
স্টাফ	ইমেইল ও পাসওয়ার্ড লগইন (Firebase Auth)	staff.html	সম্পন্ন
স্টাফ	অ্যাডভান্স বাটনসহ লাইভ অর্ডার বোর্ড	staff.html	সম্পন্ন
স্টাফ	সম্পন্ন অর্ডারের তালিকা (সর্বশেষ ১০টি)	staff.html	সম্পন্ন

এলাকা	ফিচার	কোথায়	অবস্থা
স্টাফ	মেনু: যোগ করা, এডিট, শেষ হয়ে গেছে চিহ্ন, মুছে ফেলা	staff.html	সম্পন্ন
স্টাফ	Food Waste Saver ডিসকাউন্ট (৫ থেকে ৯০ শতাংশ)	staff.html	সম্পন্ন
স্টাফ	Break Planner সেটিংস (সর্বনিম্ন, সর্বোচ্চ, ধাপ, ডিফল্ট খাওয়ার সময়)	staff.html	সম্পন্ন
স্টাফ	দিনশেষ: রিপোর্ট, অর্ডার মুছা, টোকেন রিসেট	staff.html	সম্পন্ন
স্টাফ	আগের দৈনিক রিপোর্ট (শেষ ৭টি)	staff.html	সম্পন্ন
হোম	শিক্ষার্থী ও স্টাফ রোল সুইচ	index.html	সম্পন্ন
ডিজাইন	মোবাইল-ফ্রেন্ডলি লেআউট, অ্যানিমেশন, থিম ভ্যারিয়েবল	style.css	সম্পন্ন
ভবিষ্যৎ	Green Token আর শিক্ষার্থীর দিকের Food Waste Saver	-	পরিকল্পিত
ভবিষ্যৎ	Planner-এ ভ্যারাইটি রুল, আরও কড়া নিরাপত্তা rules	-	পরিকল্পিত

5. আর্কিটেকচার ও প্রযুক্তি

স্তর	প্রযুক্তি	মন্তব্য
পেজ	HTML5, CSS3, vanilla JavaScript (ES modules)	কোনো ফ্রেমওয়ার্ক নেই, বিল্ড স্টেপ নেই
স্টাইলিং	একটাই ফাইল: :root variable-সহ style.css	এখন Spice Kitchen থিম চালু (আগে ছিল Mint)
ডাটাবেস	Cloud Firestore (asia-south1, Mumbai)	onSnapshot দিয়ে লাইভ listener
লগইন	Firebase Authentication, Email/Password	স্টাফ অ্যাকাউন্ট Firebase console থেকে বানানো
SDK	gstatic CDN থেকে Firebase Web SDK 10.12.2	ভার্সন ইচ্ছা করেই ফিক্সড রাখা
হোস্টিং	main ব্রাঞ্চ থেকে GitHub Pages, root ফোল্ডার	https://raito-del.github.io/BreakBite/
এডিটর	VS Code ও Live Server	localhost-এ পেজ টেস্ট করা হয়
ভার্সন কন্ট্রোল	Git ও GitHub Desktop	.gitattributes-এ text=auto (repo-তে LF)

প্রবাহ: শিক্ষার্থীর পেজ মেনু পড়ে আর অর্ডার লেখে। স্টাফের পেজ অর্ডার পড়ে আর স্ট্যাটাস বদলায়। দুটো পেজই একই firebase.js দিয়ে Firestore-এ যুক্ত হয়। Firestore খোলা থাকা প্রতিটা পেজে আপডেট পাঠিয়ে দেয়, তাই রিফ্রেশ করতে হয় না।

6. ফাইল তালিকা (প্রতিটা ফাইল)

এটা প্রজেক্টের root ফোল্ডার। সব ফাইল একই ফোল্ডারে থাকে, কারণ পেজগুলো relative path দিয়ে একে অপরের সাথে লিংক করে (যেমন href='student.html')।

ফাইল	কাজ	মূল বিষয়
index.html	হোম পেজ	হেডার ট্যাব, Student ও Staff রোল লিংক, হিরো লেখা, দুটো কার্ড (Order, Break Planner)
student.html	অর্ডার পেজ	Rush radar, মেনু তালিকা, কার্ট বার, অর্ডার transaction, টোকেন ট্র্যাকিং, নিজের অর্ডার, token lookup
planner.html	Break Planner	settings/planner আর মেনু একবার লোড করে, সেরা কম্বিনেশন খোঁজে, ফলাফল দেখায়
staff.html	স্টাফ ড্যাশবোর্ড	লগইন স্ক্রিন, অর্ডার বোর্ড, সম্পন্ন তালিকা, দিনশেষ, রিপোর্ট, মেনু, Food Waste Saver, সেটিংস
style.css	শেয়ার্ড stylesheet	থিম ভ্যারিয়েবল, বাটন, কার্ড, টিকিট, ধাপ, টেবিল, লগইন, responsive নিয়ম, অ্যানিমেশন, doodle ব্যাকগ্রাউন্ড
firebase.js	Firebase সংযোগ	Config object, initializeApp, db (Firestore) ও auth (Authentication) export করে
README.md	প্রজেক্টের তথ্য	নাম, বর্ণনা, ফিচার, টিম টেবিল, ফোল্ডার কাঠামো, কাজের নিয়ম, তারিখ
.gitattributes	লাইন এন্ডিং	একটাই নিয়ম: * text=auto
yellow-preview.html	থিম প্রিভিউ (সাইটে নেই)	শুরুর দিকের থিম পরীক্ষা: রং বাছাই ও প্রিসেট
spice-kitchen-preview.html	স্ট্যাটিক ডিজাইন প্রিভিউ (সাইটে নেই)	Spice Kitchen রঙে হোম, student, staff লগইন ও staff বোর্ড
spice-kitchen-animated-preview.html	অ্যানিমেটেড প্রিভিউ (সাইটে নেই)	বাটন, কার্ট, টোকেন ও ধাপের অ্যানিমেশনের ডেমো
BreakBite_Break_Planner_Presentation.pdf	শোকেস স্লাইড (সাইটে নেই)	বাংলায় ১০ স্লাইডের গল্প-ধাঁচের প্রেজেন্টেশন

নোট: ২২ অক্টোবরের আপডেটে index.html এখন স্প্ল্যাশ পেজ, আর logo.css ও logo.js নতুন ফাইল। বিস্তারিত "আপডেট ১" সেকশনে আছে।

6.1 প্রতিটা পেজের ভেতরে কী আছে (দ্রুত মানচিত্র)

- **index.html:** ডাটাবেস নেই। শুধু লিংক আর রোল আইকন। Staff লিংক staff.html খোলে, যেটা Firebase লগইন চায়।
- **student.html:** menu (order ফিল্ড অনুযায়ী সাজানো), Received বা Preparing অবস্থার orders (rush radar), আর ব্রাউজারে সেভ হওয়া প্রতিটা অর্ডার id-র (অর্ডারের অবস্থা) জন্য listener আছে। নতুন অর্ডার আর counter লেখে।
- **planner.html:** settings/planner আর menu একবারই পড়ে, লাইভ listener নেই। পুরো হিসাব ইউজারের ব্রাউজারেই হয়।
- **staff.html:** লগইন গেট; orders (সর্বশেষ ৫০টি) আর menu-র listener; স্ট্যাটাস বদল, মেনু এডিট, ডিসকাউন্ট, সেটিংস, দৈনিক রিপোর্ট আর counter রিসেট লেখে।
- **style.css:** কোনো JavaScript নেই। সব পেজ <link rel='stylesheet' href='style.css'> দিয়ে এটা ব্যবহার করে।
- **firebase.js:** কোনো UI নেই। ডাটাবেস বা লগইন ব্যবহার করা প্রতিটা পেজ এটা import করে।

7. ডেটা মডেল (Firestore)

Collection / document	ফিল্ড	কে লেখে	কে পড়ে
menu/{id}	name, price (Tk), minutes (খাওয়ার সময়), available (true/false), discount (0-90), order (সাজানোর key)	স্টাফ	শিক্ষার্থী, Planner, স্টাফ
orders/{id}	token, items [{name, qty, price}], count, total, status, createdAt	শিক্ষার্থী (তৈরি), স্টাফ (status)	শিক্ষার্থী (নিজেরটা), স্টাফ, Rush radar
counters/orders	last (সংখ্যা)	শিক্ষার্থী (transaction), স্টাফ (রিসেট)	শিক্ষার্থী (transaction)
settings/planner	minBreak, maxBreak, step, defaultEat	স্টাফ	Planner, স্টাফ
daily_reports/{id}	date, totalOrders, totalSales, items {name: qty}, activeAtClose, createdAt	স্টাফ (দিনশেষ)	স্টাফ (শেষ ৭টি)

Status-এর মান সব জায়গায় একই: Received, Preparing, Ready, Done। বোর্ডে প্রথম তিনটা দেখায়। Done হয়ে যাওয়া অর্ডার বোর্ড থেকে চলে যায় আর Completed তালিকায় আসে।

8. মূল ফিচারগুলো কীভাবে কাজ করে

8.1 অর্ডার দেওয়া (student.html)

- শিক্ষার্থী পরিমাণ বদলায়। প্রতিটা + বা - লোকাল qty object আপডেট করে, আর কার্ট বারে মোট সংখ্যা ও মোট দাম দেখায়।
- Place pre-order চাপলে একটা Firestore transaction counters/orders পড়ে, last = last + 1 করে, আর অর্ডারটা Received স্ট্যাটাসে লেখে।
- নতুন অর্ডারের id localStorage-এ (key: breakbite_orders) সেভ হয়, তাই রিফ্রেশ করলেও শিক্ষার্থী অর্ডারটা দেখতে পায়।
- শিক্ষার্থীর পেজ ওই অর্ডার ডকুমেন্টটা শোনে আর ধাপ ও বার্তা দেখায়। স্ট্যাটাস Done হলে অর্ডারটা তালিকা থেকে চলে যায়।

8.2 স্টাফের অর্ডার ফ্লো (staff.html)

- Orders listener createdAt অনুযায়ী সর্বশেষ ৫০টা অর্ডার আনে। চলমান অর্ডারগুলো তিন কলামে ভাগ হয়।
- প্রতিটা বাটন অর্ডারকে এক ধাপ এগিয়ে দেয়: Received থেকে Preparing, Preparing থেকে Ready, Ready থেকে Done।
- কিউ আর অপেক্ষার সময়: queue = Received ও Preparing অর্ডারের সংখ্যা; আনুমানিক অপেক্ষা = 3 + queue x 2 মিনিট।

8.3 Rush radar

শিক্ষার্থীর পেজ Received বা Preparing স্ট্যাটাসের অর্ডার শোনে। সংখ্যা ৫-এর কম হলে Low rush, ৫ থেকে ৯ হলে Medium rush, আর ১০ বা বেশি হলে High rush দেখায়।

8.4 Break Planner (planner.html)

Planner settings আর মেনু একবার পড়ে। বিরতির সময় minBreak আর maxBreak-এর মধ্যে আছে কিনা যাচাই করে। তারপর সময় (মিনিট) আর খরচ (Tk)-এর ওপর একটা dynamic programming টেবিল চালায়। প্রতিটা অবস্থার জন্য সবচেয়ে বেশি আইটেম সংখ্যা আর শেষ কোন আইটেমটা যোগ হয়েছে সেটা মনে রাখে। শেষে সবচেয়ে বেশি আইটেমওয়ালা অবস্থাটা বাছে (সমান হলে যেটায় খরচ বেশি), আর পেছনে হেঁটে আইটেমের তালিকা বের করে। ডিসকাউন্টের পরের দাম ধরা হয়। যে আইটেমের minutes নেই, সেখানে defaultEat ধরা হয়। মূল কোড এই ডকুমেন্টের শেষের Appendix A-তে আছে।

8.5 দিনশেষ (staff.html)

- স্টাফ End of day চাপে। দুটো নিশ্চিতকরণ প্রশ্ন আসে; দ্বিতীয়টা সাবধান করে যে এটা আর ফেরানো যাবে না।
- অ্যাপ সব অর্ডার পড়ে সারাংশ বানায় (মোট অর্ডার, মোট বিক্রি, প্রতিটা আইটেমের পরিমাণ)। এ পর্যন্ত কিছু মোছা হয় না।
- একটা daily_reports ডকুমেন্ট লেখা হয়। এই লেখা ফেল করলে পুরো প্রক্রিয়া থেমে যায় আর কোনো অর্ডার মোছা হয় না।
- অর্ডারগুলো ৪০০টা করে ব্যাচে মোছা হয়। তারপর counters/orders-এ last = 0 বসে, যাতে পরের টোকেন ১ হয়।
- আগের রিপোর্ট (শেষ ৭টি) Orders ট্যাবের নিচে দেখা যায়।

8.6 লগইন ও লগআউট

স্টাফ লগইন স্ক্রিনে ইমেইল আর পাসওয়ার্ড দেয়। Firebase সেটা যাচাই করে। onAuthStateChanged জানালে যে ইউজার সাইন-ইন করা আছে, তবেই অ্যাপ ড্যাশবোর্ড দেখায়। লগআউট বাটন signOut কল করে। পাসওয়ার্ড কোডে বা ব্রাউজারে কোথাও রাখা হয় না।

8.7 ডিসকাউন্ট (Food Waste Saver)

স্টাফ একটা খাবার আর ৫ থেকে ৯০-এর মধ্যে একটা শতাংশ বেছে নেয়। মেনু ডকুমেন্টে discount = percent বসে। শিক্ষার্থী, planner আর স্টাফ পেজ সবখানেই ডিসকাউন্টের পরের দাম দেখায়। Stop discount করলে discount আবার 0 হয়ে যায়।

9. ডিজাইন সিস্টেম ও থিম

সব রং style.css-এর :root-এ ভ্যারিয়েবল হিসেবে আছে। থিম বদলাতে শুধু এই মানগুলো বদলালেই হয়।

ভ্যারিয়েবল	Mint থিম (আগের)	Spice Kitchen (এখন চালু)	কোথায় ব্যবহার হয়
--bg	#d9f6e4	#fff3dc	পেজের ব্যাকগ্রাউন্ড
--ink	#0d3326	#3b2314	লেখা, বর্ডার, ছায়া
--accent	#1f9d6b	#d9482b	হোভার, হাইলাইট, সংখ্যা
--card	#ffffff	#ffffff	কার্ড ও হেডার
--danger	#c0392b	#b02a1a	ডিলিট ও এরর
--turmeric	(ব্যবহার হয় না)	#f2b33d	ট্যাব, ব্যাজ, কার্ট বাটন
--t-yellow / --t-blue / --t-mint	#ffe7a0 / #d4e6f7 / #c5ecd5	#ffe08a / #ffd0a8 / #cfeab0	Received / Preparing / Ready

নোট: লাইভ সাইটে এখন Spice Kitchen রং চালু আছে, style.css-এর বর্তমান কোডেও এই রংই আছে। Mint কলামটা আগের থিম।

9.1 অ্যানিমেশন

- বাটনে হোভার করলে হালকা দুলুনি আর ওপরে ওঠা; চাপলে নিচে বসে যায়।
- মেনুর সারি হোভারে একটু ওপরে ওঠে আর নরম ছায়া পায়।
- পেজ খুললে কার্ট বার নিচ থেকে স্লাইড করে ওঠে।
- চলমান ধাপ নরমভাবে লাল আভায় জ্বলে।
- Ready টোকেন ধীরে স্পন্দিত হয়। এটা Chrome ও Edge-এ :has()-এর মাধ্যমে কাজ করে।
- কোনো বাটনে loading ক্লাস থাকলে সেখানে স্পিনার ঘোরে।
- যারা reduced motion চালু রাখে, তাদের জন্য সব অ্যানিমেশন বন্ধ থাকে।

9.2 খাবারের doodle ব্যাকগ্রাউন্ড

একটা ছোট inline SVG প্যাটার্ন (বার্গার, পিৎজা, চায়ের কাপ, সমুচা, ফ্রাইজ, ড্রামস্টিক) ১০ শতাংশ স্বচ্ছতায় পেজের পেছনে থাকে। কোনো ছবির ফাইল লাগে না, তাই পেজ দ্রুত থাকে।

10. নিরাপত্তার অবস্থা ও জানা সীমাবদ্ধতা

বিষয়	বর্তমান অবস্থা	ঝুঁকি	পরিকল্পিত সমাধান
Firestore rules	১ ডিসেম্বর ২০২৬ পর্যন্ত পড়া ও লেখা সবার জন্য খোলা	যে কেউ মেনু বা অর্ডার বদলাতে পারে	menu, settings, reports, counters-এর জন্য লগইন (auth) লাগে এমন rules
স্টাফ লগইন	Firebase Email/Password	অ্যাকাউন্ট অ্যাডমিনকে যোগ করতে হয়	শুধু Email/Password রাখা; sign-up বন্ধ রাখা
হোম পেজে স্টাফ গেট	Staff আইকন লগইন পেজ খোলে	কম	এভাবেই রাখা হবে; আসল গেট হলো লগইন পেজ
Token lookup	যে কেউ টোকেন দিয়ে অর্ডার পড়তে পারে	কম (শুধু status আর items)	মেনে নেওয়া, অথবা প্রতি অর্ডারে আলাদা কোড দিয়ে সীমিত করা
Planner-এর ভ্যারাইটি	একই আইটেম বহুবার আসতে পারে	মানের সমস্যা, নিরাপত্তার নয়	প্রতি আইটেমে সর্বোচ্চ বার সীমা দেওয়া
টোকেন নম্বর	End of day পর্যন্ত গুনতে থাকে	কম	দৈনিক রিসেট ইচ্ছা করেই ম্যানুয়াল
কোডে web config	apiKey ইচ্ছা করেই পাবলিক	কম	চাবি লুকিয়ে নয়, rules দিয়ে সুরক্ষা দেওয়া

ফাইনাল শোকেসের জন্য প্রস্তাবিত rules (আগে লক্ষ্যটা লেখো, তারপর publish করো): শিক্ষার্থীরা menu ও settings পড়তে পারবে, অর্ডার তৈরি করতে পারবে এবং নিজের অর্ডার পড়তে পারবে। শুধু লগইন করা স্টাফ অর্ডার আপডেট, মেনু ও সেটিংস বদল, দৈনিক রিপোর্ট লেখা আর counter রিসেট করতে পারবে। পুরো rules এই ডকুমেন্টের শেষের Appendix A-তে আছে।

নোট: টোকেন নম্বর চালু রাখতে counters-এ শিক্ষার্থীদের create ও update করার অনুমতি লাগবে। rules বদলানোর পর প্রতিটা পেজ টেস্ট করো।

11. পুনর্নির্মাণ গাইড (ধাপে ধাপে)

শুরু থেকে BreakBite আবার বানাতে এই ধাপগুলো ফলো করো। জায়গার মানগুলো (placeholder) নিজের Firebase সেটিং দিয়ে বদলাও। আসল পাসওয়ার্ড কখনো কোডে বা এই ডকুমেন্টে লিখো না।

1. GitHub-এ একটা repository বানাও, যেমন BreakBite। একটা README.md যোগ করো।
2. সেকশন 6-এর ফাইলগুলো একই ফোল্ডারে বানাও। কোড এই ডকুমেন্টের শেষের Appendix A আর টিমের repository থেকে কপি করো।
3. console.firebase.google.com খুলে একটা প্রজেক্ট বানাও। Google Analytics বন্ধ রাখো।
4. একটা Web app যোগ করো (</> আইকন)। firebaseConfig-এর ছয়টা মান firebase.js-এ কপি করো।
5. Build > Firestore Database > Create database। Standard বেছে নাও, location asia-south1 (Mumbai), আর test mode-এ শুরু করো।
6. Firestore > Rules: এই ডকুমেন্টের শেষের Appendix A থেকে rules পেস্ট করো (ডেমোর জন্য সাময়িক খোলা rules) এবং Publish করো।
7. Build > Authentication > Get started > Sign-in method > Email/Password > Enable > Save।
8. Authentication > Users > Add user: প্রতিটা স্টাফের ইমেইল আর কমপক্ষে ৬ অক্ষরের পাসওয়ার্ড যোগ করো।
9. Authentication > Settings > Authorized domains: raito-del.github.io (বা তোমার GitHub Pages ডোমেইন) যোগ করো।
10. Firestore-এ counters নামে একটা collection বানাও। তার ভেতরে orders নামে document বানিয়ে last = 0 (number) ফিল্ড দাও।
11. GitHub-এ repo খুলে Settings > Pages > Source: Deploy from a branch > main > / (root) > Save।
12. ১ থেকে ২ মিনিট অপেক্ষা করো। https://<তোমার-ইউজার>.github.io/BreakBite/ খুলে প্রতিটা পেজ টেস্ট করো।
13. staff.html খুলে লগইন করো। নাম, দাম আর খাওয়ার সময় দিয়ে মেনু আইটেম যোগ করো, তারপর planner settings সেভ করো।
14. student.html থেকে একটা টেস্ট অর্ডার দাও। staff.html-এ সেটা এগিয়ে দাও। student.html-এ স্ট্যাটাস দেখে নাও।
15. একবার End of day চেপে রিপোর্ট টেস্ট করো, তারপর Past daily reports দেখো।

12. কাজের লগ ও টাইমলাইন

তারিখ (2026)	মাইলস্টোন
18 অক্টোবর	Repository তৈরি, প্রথম পেজগুলো, টিমের জন্য Firebase গাইড লেখা
19 অক্টোবর	Firebase প্রজেক্ট ও Firestore সেটআপ; firebase.js; প্রথম student ও staff পেজ যুক্ত করা
19 থেকে 20 অক্টোবর	Break Planner তৈরি; Firebase Auth দিয়ে স্টাফ লগইন; style.css একীভূত করা; মোবাইলের সমস্যা ঠিক করা
20 অক্টোবর	GitHub Pages লাইভ; Student ও Staff আইকনসহ হোম পেজ; token lookup; দিনশেষ ও রিপোর্ট; থিম প্রিভিউ ও অ্যানিমেশন; প্রেজেন্টেশন PDF
22 অক্টোবর	অ্যানিমেটেড লোগো, স্প্ল্যাশ পেজ (index.html), logo.css ও logo.js যোগ করা

তারিখ (2026)	মাইলস্টোন
12 নভেম্বর	ফাইনাল প্রজেক্ট জমা (ডেডলাইন)
16 নভেম্বর	শোকেস ও পোস্টার

13. ভবিষ্যতের কাজ

- Firestore rules কড়া করা (Appendix A-র লক্ষ্য অনুযায়ী) এবং দ্বিতীয় একটা staff অ্যাকাউন্ট দিয়ে টেস্ট করা।
- Break Planner-এ ভ্যারাইটি রুল যোগ করা (যেমন একটা আইটেম সর্বোচ্চ দুবার)।
- Green Token: পুনর্ব্যবহারযোগ্য কনটেইনারের জন্য শিক্ষার্থীর দিকের একটা ফিচার।
- শিক্ষার্থীর দিকের Food Waste Saver: অর্ডার পেজে ডিসকাউন্টের আইটেম স্পষ্ট ব্যাজসহ দেখানো।
- রিভিউয়ের পর Spice Kitchen থিম প্রয়োগ করা, শুধু style.css-এর রঙের ভ্যারিয়েবল বদলে (এটা এখন লাইভ সাইটে প্রয়োগ হয়ে গেছে)।
- ঐচ্ছিক: Cloud Function দিয়ে প্রতিদিন স্বয়ংক্রিয় রিসেট (Blaze প্ল্যান লাগবে)।

আপডেট ১. অ্যানিমেটেড লোগো ও স্প্ল্যাশ পেজ

তারিখ: 22 অক্টোবর 2026। এই আপডেটে BreakBite-এর লোগো আর একটা নতুন স্প্ল্যাশ পেজ যোগ হয়েছে। লোগোতে আছে একটা কুকি যেটা থেকে কামড় নেওয়া হয়েছে, কামড়ের পাশে বাইট বোঝাতে তিনটা ছোট চৌকো পিক্সেল, আর Break লেখার পাশে লাল বাটনের মতো **Bite**। রংগুলো Spice Kitchen থিমের। Firebase, ডেটা মডেল আর বাকি পেজ এই আপডেটে বদলায়নি।

U1.1 নতুন বা বদলানো ফাইল

ফাইল	কাজ	মূল বিষয়
index.html	স্প্ল্যাশ পেজ (বদলানো)	অ্যানিমেটেড কুকি লোগো, Break লেখা, লাল Bite বাটন আর "Click Bite to order food" হিন্ট। Bite বাটন চাপলে home.html খোলে। style.css ও logo.css লিংক করা আছে।
home.html	হোম পেজ	স্প্ল্যাশ পেজ যে পেজটা খোলে। এতে আগের হোম পেজের কোড থাকার কথা (এই ডকুমেন্টের Appendix B.1): হেডার ট্যাব, Student ও Staff রোল লিংক, হিরো লেখা আর দুটো কার্ড।
logo.css	লোগোর স্টাইল (নতুন)	লেআউট, Bite বাটন, হিন্ট পিল আর অ্যানিমেশন। প্রতিটা ক্লাসের নাম bb- দিয়ে শুরু, তাই style.css-এর সাথে ঝামেলা হয় না। --ink, --accent আর --gold ব্যবহার করে, না পেলে নিজের ডিফল্ট রং নেয়।
logo.js	লোগোর স্ক্রিপ্ট (নতুন)	Bite চাপলে কুকির চম্প অ্যানিমেশন আর ছিটকে পড়া টুকরো, তারপর student.html খোলে। নিচের নোট ১ দেখো।

U1.2 স্প্ল্যাশ পেজ কীভাবে কাজ করে

- পেজে কুকি, Break লেখা আর লাল Bite বাটন দেখায়। নিচে হলুদ হিন্ট পিলে লেখা "Click Bite to order food", আর তীরটা ওপরে বাটনের দিকে দেখায়।

- কুকি ভেসে ভেসে দোলে, তিনটা পিক্সেল ঝিকমিক করে, আর Bite বাটন হালকা ওঠানামা করে। হোভারে বাটন ওপরে ওঠে, চাপলে নিচে বসে যায়, সাইটের অন্য বাটনের মতোই।
- index.html-এর ভেতরের ছোট ইনলাইন স্ক্রিপ্ট Bite চাপলে home.html খুলে দেয়।
- logo.js-এ আরও বড় সংস্করণ আছে: Bite বাটন চেপ্টে যায়, কুকি চম্প করে, নয়টা টুকরো ছিটকে পড়ে, আর ০.৮ সেকেন্ড পর student.html খোলে।
- যারা reduced motion চালু রাখে, তাদের জন্য লোগোর সব অ্যানিমেশন বন্ধ থাকে।

U1.3 লোগোর ডিজাইন

- কুকিটা হলো বিরতি (break)। তার থেকে নেওয়া কামড়টা হলো নামের Bite।
- কামড়ের পাশের তিনটা চৌকো (লাল, সবুজ, হলুদ) বাইট বোঝায়, মেগাবাইট-গিগাবাইটের বাইটের মতো। 2 x 2 গ্রিডের চতুর্থ চৌকোটা নেই, যেন কামড়ে উড়ে গেছে।
- রং: কুকি #f2b33d, আউটলাইন ও ছায়া #3b2314, Bite বাটন #d9482b (ভেতরের লেখা ক্রিম #fff5e6), পিক্সেল চৌকো #cfeab0।
- স্টাইল: মোটা বাদামি আউটলাইন আর শক্ত ছায়া, সাইটের কার্ড ও বাটনের মতো।

U1.4 টিমের জন্য নোট

1. index.html এখন logo.js লোড করে না। এর নিজের ইনলাইন স্ক্রিপ্ট home.html খোলে, তাই চম্প অ্যানিমেশন আর টুকরো ছিটকানো এখনো কাজ করছে না। ওগুলো চালাতে হলে </body>-এর আগে <script src="logo.js"></script> যোগ করো, ইনলাইন স্ক্রিপ্টটা মুছে ফেলো, আর logo.js-এর TARGET-এ যে পেজ খুলতে চাও সেটার নাম দাও।
2. অন্য পেজগুলোর Home ট্যাব এখনো index.html-এ যায়, যেটা এখন স্প্ল্যাশ পেজ। Home ট্যাব হোম পেজ খুলুক চাইলে সেটাকে home.html-এ বদলাও।
3. logo.css আর logo.js প্রজেক্টের root ফোল্ডারে style.css-এর পাশেই থাকে। কোনো বিল্ড স্টেপ নেই।

শব্দের মানে (গ্লসারি)

শব্দ	মানে
Firestore	Google-এর অনলাইন ডাটাবেস, যেখানে অর্ডার আর মেনু জমা থাকে।
Firebase Authentication	লগইন ব্যবস্থা। স্টাফ ইমেইল আর পাসওয়ার্ড দিয়ে ঢোকে।
onSnapshot	ডাটাবেসে কিছু বদলালেই পেজ নিজে থেকে আপডেট হওয়ার ব্যবস্থা।
Transaction	একসাথে পড়া ও লেখার নিরাপদ ধাপ, যাতে দুজনের টোকেন নম্বর এক না হয়।
Token (টোকেন)	অর্ডারের নম্বর। খাবার রেডি হলে এটা দেখিয়ে নিতে হয়।
Rush radar	ক্যান্টিনে কয়টা অর্ডার অপেক্ষায় আছে তা দেখিয়ে ভিড় বোঝানো।
Knapsack / dynamic programming	সময় ও টাকার সীমার মধ্যে সবচেয়ে ভালো কম্বিনেশন বের করার একটা গাণিতিক পদ্ধতি।
GitHub Pages	GitHub থেকে বিনা খরচে ওয়েবসাইট চালানোর সুবিধা।

শব্দ	মানে
CDN	ইন্টারনেট থেকে সরাসরি লাইব্রেরি লোড করার ব্যবস্থা, ইনস্টল ছাড়াই।
localStorage	ব্রাউজারে ছোট তথ্য সেভ করে রাখার জায়গা।
CSS variable	একবার রং ঠিক করে নাম দিয়ে (যেমন --ink) সব জায়গায় ব্যবহার করার উপায়।
Splash page	সাইটের প্রথম স্বাগত পেজ। এখানে লোগো আর Bite বাটন আছে।
Fetch / Pull / Commit / Push	Fetch origin: GitHub-এ নতুন কিছু আছে কিনা দেখা। Pull: নতুন কাজ নামিয়ে আনা। Commit: নিজের কাজ সেভ করা। Push: সেটা GitHub-এ পাঠানো।

পরিশিষ্ট: কোড (ইংরেজিতে)

এর পরের পাতাগুলোতে প্রজেক্টের কোড আছে। কোড ইংরেজিতেই রাখা হয়েছে, আর কোডের আগে-পরের ছোট বর্ণনাগুলোও ইংরেজিতে। কোড ইংরেজিতেই লিখতে ও চালাতে হয়, তাই ওগুলো বাংলা করা হয়নি।

অংশ	কী আছে
Appendix A. Key source code	সবচেয়ে গুরুত্বপূর্ণ কোডগুলো: firebase.js (সংযোগ ফাইল), নিরাপদ টোকেনসহ অর্ডার দেওয়া, Planner-এর সেরা কম্বিনেশন বের করা, দিনশেষের রিপোর্ট ও রিসেট, আর Firestore rules (এখনকার খোলা rules ও লক্ষ্য rules)।
Appendix B. B.1 থেকে B.8	প্রতিটা ফাইলের পূর্ণ কোড: index.html (আগের হোম পেজ), firebase.js, student.html, planner.html, staff.html, style.css, README.md, .gitattributes।
Appendix B. B.9 থেকে B.11	22 অক্টোবরের নতুন ফাইল: index.html (নতুন স্প্ল্যাশ পেজ), logo.css, logo.js।

- B.1-এর index.html হলো আগের হোম পেজের কোড। নতুন স্প্ল্যাশ পেজ চালু করলে এই আগের কোডটা home.html নামে রাখতে হবে (আপডেট ১ দেখো)।
- firebase.js-এ আসল Firebase কী-এর বদলে YOUR_API_KEY-এর মতো placeholder আছে। নিজের মানগুলো Firebase console থেকে নিয়ে বসাতে হবে।

Appendix A. Key source code

The full, current source of every page is in the GitHub repository. The excerpts below are the parts that carry the important logic. Placeholder values replace the real Firebase keys.

A.1 firebase.js (connection file, placeholders only)

```
// firebase.js: the only place where the Firebase settings live.
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-app.js";
import { getFirestore } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";
import { getAuth } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-auth.js";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT_ID.firebaseapp.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT_ID.firebasestorage.app",
  messagingSenderId: "YOUR_SENDER_ID",
  appId: "YOUR_APP_ID"
};

const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
export const auth = getAuth(app);
```

A.2 Placing an order with a safe token (student.html)

```
const counterRef = doc(db, "counters", "orders");
const orderRef = doc(collection(db, "orders"));
await runTransaction(db, async function (tx) {
  const cs = await tx.get(counterRef);
  const next = (cs.exists() ? cs.data().last : 0) + 1;   // next token number
  tx.set(counterRef, { last: next });
  tx.set(orderRef, {
    token: next, items: items, count: count, total: total,
    status: "Received", createdAt: serverTimestamp()
  });
});
addId(orderRef.id);   // remember this order in this browser
```

A.3 Planner: best combination (planner.html)

```
function bestCombo(minutes, money) {
  const cap = Math.min(money, 1500);
  const W = cap + 1;
  const size = (minutes + 1) * W;
  const count = new Int16Array(size).fill(-1);  // -1 means not reachable
  const from = new Int16Array(size).fill(-1);   // which food was added last
  count[0] = 0;
  for (let i = 0; i < foods.length; i++) {      // unbounded knapsack
    const ft = foods[i].minutes, fp = foods[i].price;
    if (ft > minutes || fp > cap) continue;
    for (let t = ft; t <= minutes; t++) {
      for (let m = fp; m <= cap; m++) {
        const prev = (t - ft) * W + (m - fp);
        if (count[prev] < 0) continue;
        const idx = t * W + m;
        if (count[prev] + 1 > count[idx]) { count[idx] = count[prev] + 1; from[idx] = i; }
      }
    }
  }
  // pick the state with most items, then the bigger spend
  let bestIdx = 0, bestCount = 0, bestSpend = 0;
  for (let t = 0; t <= minutes; t++) for (let m = 0; m <= cap; m++) {
    const c = count[t * W + m];
    if (c > bestCount || (c === bestCount && c > 0 && m > bestSpend)) {
      bestCount = c; bestSpend = m; bestIdx = t * W + m;
    }
  }
  // walk back to list the chosen items
  const picked = []; let idx = bestIdx, guard = 0;
  while (idx > 0 && from[idx] >= 0 && guard++ < 200) {
    const f = foods[from[idx]]; picked.push(f);
    const t = Math.floor(idx / W), m = idx % W;
    idx = (t - f.minutes) * W + (m - f.price);
  }
  return picked;
}
```

A.4 End of day: report first, delete second (staff.html)

```
await addDoc(collection(db, "daily_reports"), {        // 1. save report first
  date: todayKey(), totalOrders: all.size, totalSales: totalSales,
  items: itemCount, activeAtClose: activeCount, createdAt: serverTimestamp()
});
const docs = all.docs;                                  // 2. delete in batches of 400
for (let i = 0; i < docs.length; i += 400) {
  const batch = writeBatch(db);
  docs.slice(i, i + 400).forEach(function (d) { batch.delete(d.ref); });
  await batch.commit();
}
await setDoc(doc(db, "counters", "orders"), { last: 0 });   // 3. restart tokens
```

A.5 Current rules (open, demo only)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if request.time < timestamp.date(2026, 12, 1);
    }
  }
}
```

A.6 Target rules for the showcase (recommended)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isStaff() { return request.auth != null; }

    match /menu/{id}      { allow read: if true;  allow write: if isStaff(); }
    match /settings/{id}  { allow read: if true;  allow write: if isStaff(); }
    match /counters/{id}  { allow read: if true;  allow write: if true; }
    match /orders/{id} {
      allow read: if true;
      allow create: if true;
      allow update, delete: if isStaff();
    }
    match /daily_reports/{id} { allow read: if isStaff(); allow create: if isStaff(); }
  }
}
```

Note: counters must allow create and update from students for token numbers to work. Test every page after changing rules.

Appendix B. Complete source of every file

This appendix contains the full text of each project file as it is in the repository. The Firebase settings in firebase.js are replaced by placeholders on purpose, so this document is safe to share. Put your own values from the Firebase console into your local firebase.js.

Order	File	Lines (approx.)
B.1	index.html	home page
B.2	firebase.js	Firebase connection (placeholders)
B.3	student.html	ordering page
B.4	planner.html	break planner
B.5	staff.html	staff dashboard
B.6	style.css	shared stylesheet, Spice Kitchen theme
B.7	README.md	project information
B.8	.gitattributes	line endings

B.1 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BreakBite</title>
  <link rel="stylesheet" href="style.css">
  <style>
    .role-switch { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 20px; }
    .role-btn {
      display: flex; align-items: center; gap: 10px;
      background: var(--card); color: var(--ink);
      border: 2px solid var(--ink); border-radius: 12px;
      padding: 12px 18px; box-shadow: var(--shadow-small);
      font-weight: 800; font-size: 16px; cursor: pointer; text-decoration: none;
    }
    .role-btn .ico {
      display: inline-grid; place-items: center;
      width: 34px; height: 34px; border-radius: 9px;
      background: var(--ink); color: var(--card); font-size: 18px;
    }
    .role-btn.on { background: var(--t-mint); }
    .role-btn:hover { background: var(--accent); color: #fff; }
    .role-btn:hover .ico { background: #fff; color: var(--ink); }
  </style>
</head>
<body>

<header class="top">
  <div class="brand">BreakBite</div>
  <nav class="tabs" aria-label="Main sections">
    <a class="tab" href="student.html">Order</a>
    <a class="tab" href="planner.html">Break Planner</a>
    <a class="tab active" href="index.html">Home</a>
  </nav>
</header>

<main class="wide">
  <div class="role-switch">
    <a class="role-btn on" href="index.html">
      <span class="ico">S</span> Student
    </a>
    <a class="role-btn" href="staff.html">
      <span class="ico">T</span> Staff
    </a>
  </div>

  <section class="hero">
    <h1>Less waiting, more break.</h1>
    <p>Pre-order your canteen food, skip the queue, and plan your break with the time you have.</p>
  </section>

  <div class="grid">
    <a class="card" href="student.html">
      <h2>Order food</h2>
      <p>See today's menu, pick your items, and track your token until it is ready.</p>
      <span class="go">Open ordering &rarr;</span>
    </a>
    <a class="card" href="planner.html">
      <h2>Break Planner</h2>
      <p>Enter your break time and money. We suggest what you can eat in that time.</p>
      <span class="go">Plan my break &rarr;</span>
    </a>
  </div>
</main>

</body>
</html>
```

B.2 firebase.js (placeholders)

Replace the placeholder values with the six values from your Firebase web app settings.

```
// firebase.js: the only place where the Firebase settings live.
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-app.js";
import { getFirestore } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";
import { getAuth } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-auth.js";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT_ID.firebaseapp.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT_ID.firebasestorage.app",
  messagingSenderId: "YOUR_SENDER_ID",
  appId: "YOUR_APP_ID"
};

const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
export const auth = getAuth(app);
```

B.3 student.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BreakBite - Order</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header class="top">
  <div class="brand">BreakBite</div>
  <nav class="tabs" aria-label="Student sections">
    <a class="tab active" href="student.html">Order</a>
    <a class="tab" href="planner.html">Break Planner</a>
    <a class="tab" href="index.html">Home</a>
  </nav>
</header>

<main class="narrow">
  <section class="hero">
    <h1>Pre-order, skip the queue.</h1>
    <p class="muted">Smart Food, Smart Time. Pick your food, place the order, and collect it when your
        token is ready.</p>
  </section>

  <div class="card rush">
    <div>
      <b>Canteen rush</b>
      <div class="muted" id="rush-text">Checking...</div>
    </div>
    <div class="num" id="rush-num">-</div>
  </div>

  <div class="card">
    <h2>Today's menu</h2>
    <div id="menu-list"><div class="empty">Loading menu...</div></div>
  </div>

  <div class="cart">
    <span id="cart-text">No items selected</span>
    <button id="order-btn" disabled>Place pre-order</button>
  </div>

  <div class="card" id="orders-card" hidden>
    <h2>Your orders</h2>
    <div id="my-orders"></div>
  </div>

  <div class="card">
    <h2>Check a token</h2>
    <p class="muted">Forgot your status? Enter your token number.</p>
    <div class="form-row">
      <input type="number" id="lookup-token" placeholder="Token number" min="1">
      <button id="lookup-btn">Check status</button>
    </div>
    <div id="lookup-result"></div>
  </div>
</main>

<div id="toast" hidden></div>

<script type="module">
import { db } from "./firebase.js";
import {
  collection, doc, onSnapshot, query, orderBy, where, getDocs, runTransaction, serverTimestamp
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";

const STEPS = ["Received", "Preparing", "Ready"];
const KEY = "breakbite_orders";      // list of order IDs this browser placed
const OLD_KEY = "breakbite_order";   // single ID used by the earlier version

let menu = [];
let qty = {};
let pending = 0;
let live = {};       // orderId -> latest order data
let watching = {};   // orderId -> unsubscribe function
```

```
const $ = function (id) { return document.getElementById(id); };

function esc(s) {
  return String(s).replace(/[&<>"']/g, function (c) {
    return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c];
  });
}

function price(f) {
  return Math.round(f.price * (100 - (f.discount || 0)) / 100);
}

function toast(msg) {
  const t = $("toast");
  t.textContent = msg;
  t.hidden = false;
  clearTimeout(toast.timer);
  toast.timer = setTimeout(function () { t.hidden = true; }, 2500);
}

function waitMinutes() {
  return 3 + pending * 2;
}

function cartItems() {
  return menu
    .filter(function (f) { return qty[f.id] > 0; })
    .map(function (f) { return { name: f.name, qty: qty[f.id], price: price(f) }; });
}

// ---------- saved order IDs ----------
function loadIds() {
  try { return JSON.parse(localStorage.getItem(KEY)) || []; } catch (e) { return []; }
}
function saveIds(ids) {
  localStorage.setItem(KEY, JSON.stringify(ids));
}
function addId(id) {
  const ids = loadIds();
  if (ids.indexOf(id) < 0) ids.push(id);
  saveIds(ids);
}
function removeId(id) {
  saveIds(loadIds().filter(function (x) { return x !== id; }));
  if (watching[id]) { watching[id](); delete watching[id]; }
  delete live[id];
  renderMyOrders();
}

// Move an order saved by the earlier version into the new list
(function migrate() {
  const old = localStorage.getItem(OLD_KEY);
  if (old) {
    addId(old);
    localStorage.removeItem(OLD_KEY);
  }
})();

// ---------- status helpers ----------
function stepsHtml(status) {
  const idx = status === "Done" ? STEPS.length : STEPS.indexOf(status);
  return '<div class="steps">' + STEPS.map(function (s, i) {
    const cls = i < idx ? " done" : (i === idx ? " now" : "");
    return '<div class="step' + cls + '"><span>' + (i + 1) + '</span>' + s + '</div>';
  }).join("") + '</div>';
}

function statusMessage(status) {
  if (status === "Ready") return "Ready! Please collect your food.";
  if (status === "Preparing") return "Your food is being prepared.";
  if (status === "Done") return "Thank you! Order collected.";
  return "Waiting. Estimated " + waitMinutes() + " min.";
}

function itemsText(o) {
  return o.items.map(function (i) { return i.name + " x" + i.qty; }).join(", ");
}
```

```
// ---------- Menu ----------
onSnapshot(query(collection(db, "menu"), orderBy("order")),
  function (snap) {
    menu = snap.docs
      .map(function (d) { return Object.assign({ id: d.id }, d.data()); })
      .filter(function (f) { return f.available !== false; });
    drawMenu();
    updateCart();
  },
  function (err) {
    $("menu-list").innerHTML = '<div class="empty">Could not load the menu. Check the Firebase
        setup.</div>';
    console.error(err);
  }
);

function drawMenu() {
  if (!menu.length) {
    $("menu-list").innerHTML = '<div class="empty">No items right now. Please check again later.</div>';
    return;
  }
  $("menu-list").innerHTML = menu.map(function (f) {
    const p = price(f);
    const priceHtml = f.discount > 0
      ? '<s class="muted">' + f.price + ' Tk</s> <b>' + p + ' Tk</b> <span class="badge">' + f.discount
          + '% off</span>'
      : '<b>' + f.price + ' Tk</b>';
    const q = qty[f.id] || 0;
    return '<div class="item">' +
      '<div><div class="name">' + esc(f.name) + '</div><div>' + priceHtml + '</div></div>' +
      '<div class="stepper">' +
        '<button class="ghost small" data-act="dec" data-id="' + f.id + '">-</button>' +
        '<span>' + q + '</span>' +
        '<button class="small" data-act="inc" data-id="' + f.id + '">+</button>' +
      '</div></div>';
  }).join("");
}

$("menu-list").addEventListener("click", function (e) {
  const b = e.target.closest("button[data-act]");
  if (!b) return;
  const id = b.dataset.id;
  const cur = qty[id] || 0;
  qty[id] = b.dataset.act === "inc" ? Math.min(cur + 1, 20) : Math.max(cur - 1, 0);
  if (qty[id] === 0) delete qty[id];
  drawMenu();
  updateCart();
});

function updateCart() {
  const items = cartItems();
  const count = items.reduce(function (a, i) { return a + i.qty; }, 0);
  const total = items.reduce(function (a, i) { return a + i.qty * i.price; }, 0);
  $("cart-text").textContent = count ? count + " item(s), " + total + " Tk" : "No items selected";
  $("order-btn").disabled = count === 0;
}

// ---------- Rush radar ----------
onSnapshot(query(collection(db, "orders"), where("status", "in", ["Received", "Preparing"])),
  function (snap) {
    pending = snap.size;
    let text;
    if (pending >= 10) text = "High rush. Pre-order to save time.";
    else if (pending >= 5) text = "Medium rush. Pre-order if you can.";
    else text = "Low rush. Good time to visit.";
    $("rush-text").textContent = text + " " + pending + " order(s) waiting.";
    $("rush-num").textContent = pending;
  },
  function (err) { console.error(err); }
);

// ---------- Your orders (all active orders from this browser) ----------
function renderMyOrders() {
  const ids = Object.keys(live);
  $("orders-card").hidden = ids.length === 0;
  $("my-orders").innerHTML = ids.map(function (id) {
    const o = live[id];
    return '<div class="card">' +
```

```
      '<div class="ticket-big">' +
        '<small>Your token</small>' +
        '<div class="tok">' + o.token + '</div>' +
        '<div class="muted">' + esc(itemsText(o)) + '</div>' +
      '</div>' +
      stepsHtml(o.status) +
      '<p class="track-msg">' + statusMessage(o.status) + '</p>' +
    '</div>';
  }).join("");
}

function watch(id) {
  if (watching[id]) return;
  watching[id] = onSnapshot(doc(db, "orders", id),
    function (snap) {
      if (!snap.exists()) { removeId(id); return; }
      const o = snap.data();
      if (o.status === "Done") {
        toast("Token " + o.token + " collected. Thank you!");
        removeId(id);
      } else {
        live[id] = o;
        renderMyOrders();
      }
    },
    function (err) { console.error(err); }
  );
}

// ---------- Place order ----------
$("order-btn").addEventListener("click", async function () {
  const items = cartItems();
  if (!items.length) return;
  const count = items.reduce(function (a, i) { return a + i.qty; }, 0);
  const total = items.reduce(function (a, i) { return a + i.qty * i.price; }, 0);

  const btn = $("order-btn");
  btn.disabled = true;
  btn.textContent = "Placing...";
  try {
    const counterRef = doc(db, "counters", "orders");
    const orderRef = doc(collection(db, "orders"));
    await runTransaction(db, async function (tx) {
      const cs = await tx.get(counterRef);
      const next = (cs.exists() ? cs.data().last : 0) + 1;
      tx.set(counterRef, { last: next });
      tx.set(orderRef, {
        token: next,
        items: items,
        count: count,
        total: total,
        status: "Received",
        createdAt: serverTimestamp()
      });
    });
    addId(orderRef.id);
    qty = {};
    drawMenu();
    updateCart();
    watch(orderRef.id);
    toast("Order placed!");
  } catch (err) {
    toast("Could not place the order. Check your internet and try again.");
    console.error(err);
  }
  btn.textContent = "Place pre-order";
  updateCart();
});

// ---------- Check any token ----------
$("lookup-btn").addEventListener("click", async function () {
  const n = parseInt($("lookup-token").value, 10);
  if (isNaN(n) || n < 1) { toast("Enter a valid token number."); return; }
  try {
    const snap = await getDocs(query(collection(db, "orders"), where("token", "==", n)));
    if (snap.empty) {
      $("lookup-result").innerHTML = '<p class="muted">Token ' + n + ' not found.</p>';
      return;
```

```
    }
    const o = snap.docs[0].data();
    $("lookup-result").innerHTML =
      '<div class="ticket-big"><small>Token</small><div class="tok">' + o.token + '</div>' +
      '<div class="muted">' + esc(itemsText(o)) + '</div></div>' +
      stepsHtml(o.status) +
      '<p class="track-msg">' + statusMessage(o.status) + '</p>';
  } catch (err) {
    toast("Could not check the token. Try again.");
    console.error(err);
  }
});

// ---------- Start: reconnect every order saved in this browser ----------
loadIds().forEach(watch);
</script>
</body>
</html>
```

B.4 planner.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BreakBite - Break Planner</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header class="top">
  <div class="brand">BreakBite</div>
  <nav class="tabs" aria-label="Main sections">
    <a class="tab" href="student.html">Order</a>
    <a class="tab active" href="planner.html">Break Planner</a>
    <a class="tab" href="index.html">Home</a>
  </nav>
</header>

<main class="narrow">
  <h1>Break Planner</h1>
  <p class="intro">Tell us how long your break is and how much money you have. We will suggest what you
      can eat.</p>

  <div class="card">
    <h2>Your break</h2>
    <div class="form-grid">
      <label>Break time (minutes)
        <input type="number" id="time" placeholder="e.g. 15">
      </label>
      <label>Money (Tk)
        <input type="number" id="money" min="0" placeholder="e.g. 100">
      </label>
    </div>
    <button id="plan-btn" disabled>Loading menu...</button>
    <p class="error" id="error" role="alert"></p>
  </div>

  <div class="card" id="result-card" hidden>
    <h2>Suggested for you</h2>
    <div class="summary">
      <span id="result-info"></span>
      <b id="result-total"></b>
    </div>
    <div id="result-list"></div>
  </div>
</main>

<script type="module">
import { db } from "./firebase.js";
import {
  collection, doc, getDoc, getDocs, query, orderBy
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";

const DEFAULTS = { minBreak: 10, maxBreak: 45, step: 5, defaultEat: 5 };

let settings = Object.assign({}, DEFAULTS);
let foods = [];

const $ = function (id) { return document.getElementById(id); };

function esc(s) {
  return String(s).replace(/[&<>"']/g, function (c) {
    return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c];
  });
}

function showError(msg) {
  $("error").textContent = msg;
  if (msg) $("result-card").hidden = true;
}

function finalPrice(f) {
  return Math.round(f.price * (100 - (f.discount || 0)) / 100);
}

// ---------- load settings and menu once ----------
async function load() {
```

```
  try {
    const sSnap = await getDoc(doc(db, "settings", "planner"));
    if (sSnap.exists()) settings = Object.assign({}, DEFAULTS, sSnap.data());
  } catch (err) {
    console.error(err); // fall back to defaults
  }

  try {
    const snap = await getDocs(query(collection(db, "menu"), orderBy("order")));
    foods = snap.docs
      .map(function (d) { return Object.assign({ id: d.id }, d.data()); })
      .filter(function (f) { return f.available !== false; })
      .map(function (f) {
        return {
          name: f.name,
          price: finalPrice(f),
          oldPrice: f.price,
          discount: f.discount || 0,
          minutes: f.minutes > 0 ? f.minutes : settings.defaultEat
        };
      })
      .filter(function (f) { return f.price >= 0 && f.minutes > 0; });
  } catch (err) {
    console.error(err);
    showError("Could not load the menu. Check your internet and refresh.");
    $("plan-btn").textContent = "Show options";
    return;
  }

  const t = $("time");
  t.min = settings.minBreak;
  t.max = settings.maxBreak;
  t.step = settings.step;
  t.placeholder = "e.g. " + Math.min(settings.maxBreak, Math.max(settings.minBreak, 15));

  $("plan-btn").disabled = false;
  $("plan-btn").textContent = "Show options";
  if (!foods.length) showError("No food is available right now. Please check again later.");
}

// ---------- best combination ----------
// Most items that fit the time and money. Ties go to the bigger spend.
// Same item can be picked more than once.
function bestCombo(minutes, money) {
  const cap = Math.min(money, 1500);
  const W = cap + 1;
  const size = (minutes + 1) * W;
  const count = new Int16Array(size).fill(-1);
  const from = new Int16Array(size).fill(-1);
  count[0] = 0;

  for (let i = 0; i < foods.length; i++) {
    const ft = foods[i].minutes, fp = foods[i].price;
    if (ft > minutes || fp > cap) continue;
    for (let t = ft; t <= minutes; t++) {
      for (let m = fp; m <= cap; m++) {
        const prev = (t - ft) * W + (m - fp);
        if (count[prev] < 0) continue;
        const idx = t * W + m;
        if (count[prev] + 1 > count[idx]) {
          count[idx] = count[prev] + 1;
          from[idx] = i;
        }
      }
    }
  }

  let bestIdx = 0, bestCount = 0, bestSpend = 0;
  for (let t = 0; t <= minutes; t++) {
    for (let m = 0; m <= cap; m++) {
      const c = count[t * W + m];
      if (c > bestCount || (c === bestCount && c > 0 && m > bestSpend)) {
        bestCount = c; bestSpend = m; bestIdx = t * W + m;
      }
    }
  }

  const picked = [];
```

```
  let idx = bestIdx;
  let guard = 0;
  while (idx > 0 && from[idx] >= 0 && guard++ < 200) {
    const f = foods[from[idx]];
    picked.push(f);
    const t = Math.floor(idx / W), m = idx % W;
    idx = (t - f.minutes) * W + (m - f.price);
  }
  return picked;
}

function groupItems(items) {
  const map = {};
  items.forEach(function (f) {
    if (!map[f.name]) map[f.name] = { food: f, qty: 0 };
    map[f.name].qty++;
  });
  return Object.keys(map).map(function (k) { return map[k]; });
}

$("plan-btn").addEventListener("click", function () {
  const minutes = parseInt($("time").value, 10);
  const money = parseInt($("money").value, 10);

  if (isNaN(minutes) || minutes < settings.minBreak || minutes > settings.maxBreak) {
    showError("Please enter a break time between " + settings.minBreak + " and " + settings.maxBreak + "
        minutes.");
    return;
  }
  if (isNaN(money) || money < 0) {
    showError("Please enter how much money you have.");
    return;
  }
  if (!foods.length) {
    showError("No food is available right now. Please check again later.");
    return;
  }
  showError("");

  const items = bestCombo(minutes, money);
  const list = $("result-list");

  if (!items.length) {
    list.innerHTML = '<div class="empty">Nothing fits in ' + minutes + ' minutes and ' + money +
      ' Tk. Try more time or money.</div>';
    $("result-info").textContent = minutes + " min, " + money + " Tk";
    $("result-total").textContent = "";
  } else {
    const totalTime = items.reduce(function (a, f) { return a + f.minutes; }, 0);
    const totalCost = items.reduce(function (a, f) { return a + f.price; }, 0);
    list.innerHTML = groupItems(items).map(function (g) {
      const f = g.food;
      const off = f.discount > 0 ? ' <span class="badge">' + f.discount + '% off</span>' : '';
      return '<div class="item"><div><div class="name">' + esc(f.name) + ' x' + g.qty + off +
        '</div><div class="meta">' + f.minutes + ' min each</div></div>' +
        '<div class="price">' + (f.price * g.qty) + ' Tk</div></div>';
    }).join("");
    $("result-info").textContent = "Time used: " + totalTime + " of " + minutes + " min";
    $("result-total").textContent = "Total: " + totalCost + " Tk";
  }
  $("result-card").hidden = false;
});

load();
</script>

</body>
</html>
```

B.5 staff.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BreakBite Staff</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<!-- ================= LOGIN ================= -->
<div id="login-screen">
  <div class="login-brand">
    <div class="logo">BreakBite</div>
    <p>Smart Food, Smart Time. Manage your canteen orders and menu from one place.</p>
  </div>
  <div class="login-form">
    <form id="login-form">
      <h2>Staff login</h2>
      <p class="muted">Only canteen staff can open this page.</p>
      <label class="field">Email
        <input type="email" id="username" autocomplete="username" required>
      </label>
      <label class="field">Password
        <input type="password" id="password" autocomplete="current-password" required>
      </label>
      <div class="msg" id="login-error" role="alert"></div>
      <button type="submit">Log in</button>
    </form>
  </div>
</div>

<!-- ================= APP ================= -->
<div id="app" class="hidden">
  <header class="top">
    <div class="brand">BreakBite</div>
    <nav class="tabs" aria-label="Staff sections">
      <button class="tab active" data-tab="orders">Orders</button>
      <button class="tab" data-tab="menu">Menu</button>
      <button class="tab" data-tab="waste">Food Waste Saver</button>
      <button class="tab" data-tab="settings">Settings</button>
    </nav>
    <div class="right"><button class="ghost small" id="logout-btn">Log out</button></div>
  </header>

  <main>
    <!-- Orders -->
    <section class="panel active" id="panel-orders">
      <h2>Orders</h2>
      <div class="stats">
        <div class="stat" id="wait-stat">
          <div class="muted">Orders in queue</div>
          <div class="num" id="queue-count">0</div>
        </div>
        <div class="stat">
          <div class="muted">Estimated waiting time</div>
          <div class="num" id="wait-time">3 min</div>
        </div>
        <div class="stat">
          <div class="muted">Ready to collect</div>
          <div class="num" id="ready-count">0</div>
        </div>
      </div>

      <div class="board">
        <div class="col">
          <h3>Received <span class="count" id="cnt-Received">0</span></h3>
          <div id="col-Received"></div>
        </div>
        <div class="col">
          <h3>Preparing <span class="count" id="cnt-Preparing">0</span></h3>
          <div id="col-Preparing"></div>
        </div>
        <div class="col">
          <h3>Ready <span class="count" id="cnt-Ready">0</span></h3>
          <div id="col-Ready"></div>
        </div>
      </div>
```

```
      <div class="card">
        <h3>Completed (latest 10)</h3>
        <div id="done-list"></div>
      </div>

      <div class="card">
        <h3>End of day</h3>
        <p class="muted">Saves today's report, removes all orders, and restarts token numbers from 1.
            Press this once at the end of the day.</p>
        <button class="danger" id="eod-btn">End of day: save report and reset tokens</button>
      </div>

      <div class="card">
        <h3>Past daily reports</h3>
        <div id="reports-list"></div>
      </div>
    </section>

    <!-- Menu -->
    <section class="panel" id="panel-menu">
      <h2>Menu</h2>
      <div class="card">
        <div class="table-wrap">
          <table>
            <thead>
              <tr><th>Item</th><th>Price</th><th>Eat time</th><th>Status</th><th>Actions</th></tr>
            </thead>
            <tbody id="menu-body"></tbody>
          </table>
        </div>
      </div>
      <div class="card">
        <h3>Add a new item</h3>
        <div class="form-row">
          <input type="text" id="food-name" placeholder="Item name">
          <input type="number" id="food-price" placeholder="Price (Tk)" min="0">
          <input type="number" id="food-minutes" placeholder="Eat time (min)" min="1">
          <button id="add-food-btn">Add item</button>
        </div>
      </div>
    </section>

    <!-- Food Waste Saver -->
    <section class="panel" id="panel-waste">
      <h2>Food Waste Saver</h2>
      <div class="card">
        <h3>Start a discount</h3>
        <div class="form-row">
          <select id="waste-food"></select>
          <input type="number" id="waste-percent" placeholder="Discount %" min="5" max="90">
          <button id="waste-start">Start discount</button>
        </div>
      </div>
      <div class="card">
        <h3>Running discounts</h3>
        <div id="waste-list"></div>
      </div>
    </section>

    <!-- Settings -->
    <section class="panel" id="panel-settings">
      <h2>Settings</h2>

      <div class="card">
        <h3>Break Planner</h3>
        <p class="muted">These values control the Break Planner page for students.</p>
        <div class="grid2">
          <label class="field">Shortest break (min)
            <input type="number" id="pl-min" min="1">
          </label>
          <label class="field">Longest break (min)
            <input type="number" id="pl-max" min="1">
          </label>
          <label class="field">Time step (min)
            <input type="number" id="pl-step" min="1">
          </label>
          <label class="field">Default eat time if missing (min)
```

```
            <input type="number" id="pl-default" min="1">
          </label>
        </div>
        <button id="planner-save">Save planner settings</button>
      </div>

      <div class="card">
        <h3>Change password</h3>
        <p class="muted">Password is managed through Firebase Authentication. Use the Firebase console
            to reset it.</p>
      </div>
    </section>
  </main>
</div>

<div id="toast" class="hidden"></div>

<script type="module">
import { db, auth } from "./firebase.js";
import {
  collection, doc, addDoc, updateDoc, deleteDoc, setDoc, getDoc, getDocs,
  onSnapshot, query, orderBy, limit, writeBatch, serverTimestamp
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";
import {
  signInWithEmailAndPassword, signOut, onAuthStateChanged
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-auth.js";

const STATUS_FLOW = ["Received", "Preparing", "Ready", "Done"];
const NEXT_LABEL = { Received: "Start preparing", Preparing: "Mark ready", Ready: "Mark collected" };
const DEFAULT_EAT_MIN = 5;

let menu = [];
let orders = [];
let started = false;

const $ = function (id) { return document.getElementById(id); };

function escapeHtml(s) {
  return String(s).replace(/[&<>"']/g, function (c) {
    return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c];
  });
}

function toast(msg) {
  const t = $("toast");
  t.textContent = msg;
  t.classList.remove("hidden");
  clearTimeout(toast.timer);
  toast.timer = setTimeout(function () { t.classList.add("hidden"); }, 2500);
}

function finalPrice(f) {
  return Math.round(f.price * (100 - (f.discount || 0)) / 100);
}

function itemsText(o) {
  return o.items.map(function (i) { return i.name + " x" + i.qty; }).join(", ");
}

function timeText(o) {
  const d = o.createdAt && o.createdAt.toDate ? o.createdAt.toDate() : new Date();
  return d.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" });
}

function todayKey() {
  return new Date().toLocaleDateString("en-CA"); // YYYY-MM-DD in local time
}

// ---------- login ----------
onAuthStateChanged(auth, function (user) {
  if (user) {
    $("login-screen").classList.add("hidden");
    $("app").classList.remove("hidden");
    if (!started) {
      started = true;
      watchOrders();
      watchMenu();
      watchReports();
```

```
      loadPlannerSettings();
    }
  } else {
    $("login-screen").classList.remove("hidden");
    $("app").classList.add("hidden");
  }
});

$("login-form").addEventListener("submit", async function (e) {
  e.preventDefault();
  $("login-error").textContent = "";
  try {
    await signInWithEmailAndPassword(auth, $("username").value.trim(), $("password").value);
    $("password").value = "";
  } catch (err) {
    $("login-error").textContent = "Wrong email or password. Try again.";
    console.error(err);
  }
});

$("logout-btn").addEventListener("click", function () { signOut(auth); });

// ---------- tabs ----------
function showTab(name) {
  document.querySelectorAll(".tab").forEach(function (b) {
    b.classList.toggle("active", b.dataset.tab === name);
  });
  document.querySelectorAll(".panel").forEach(function (p) {
    p.classList.toggle("active", p.id === "panel-" + name);
  });
}
document.querySelectorAll(".tab").forEach(function (b) {
  b.addEventListener("click", function () { showTab(b.dataset.tab); });
});

// ---------- one click handler for dynamic buttons ----------
document.addEventListener("click", function (e) {
  const el = e.target.closest("[data-act]");
  if (!el) return;
  const id = el.dataset.id;
  const act = el.dataset.act;
  if (act === "advance") advanceOrder(id);
  if (act === "toggle") toggleAvailable(id);
  if (act === "edit") editFood(id);
  if (act === "delete") deleteFood(id);
  if (act === "stopDiscount") setDiscount(id, 0);
});

// ---------- orders ----------
function watchOrders() {
  onSnapshot(query(collection(db, "orders"), orderBy("createdAt", "desc"), limit(50)),
    function (snap) {
      orders = snap.docs.map(function (d) { return Object.assign({ id: d.id }, d.data()); });
      renderOrders();
    },
    function (err) { toast("Could not load orders."); console.error(err); }
  );
}

function renderOrders() {
  const active = orders.filter(function (o) { return o.status !== "Done"; });
  const queue = active.filter(function (o) { return o.status === "Received" || o.status === "Preparing";
      }).length;
  const ready = active.filter(function (o) { return o.status === "Ready"; }).length;

  $("queue-count").textContent = queue;
  $("wait-time").textContent = (3 + queue * 2) + " min";
  $("ready-count").textContent = ready;
  $("wait-stat").classList.toggle("hot", queue >= 8);

  ["Received", "Preparing", "Ready"].forEach(function (s) {
    const list = active.filter(function (o) { return o.status === s; });
    $("cnt-" + s).textContent = list.length;
    $("col-" + s).innerHTML = list.length
      ? list.map(ticketHtml).join("")
      : '<div class="empty">No ' + s.toLowerCase() + ' orders.</div>';
  });
```

```
  const done = orders.filter(function (o) { return o.status === "Done"; }).slice(0, 10);
  $("done-list").innerHTML = done.length
    ? done.map(function (o) {
        return '<div class="list-row">' +
          '<span><span class="badge off">#' + o.token + '</span> ' + escapeHtml(itemsText(o)) +
              '</span>' +
          '<span class="muted">' + o.total + ' Tk, ' + timeText(o) + '</span>' +
        '</div>';
      }).join("")
    : '<div class="empty">Nothing completed yet.</div>';
}

function ticketHtml(o) {
  return '<div class="ticket ' + o.status + '">' +
    '<div class="stub"><small>Token</small><b>' + o.token + '</b></div>' +
    '<div class="body">' +
      '<div class="items">' + escapeHtml(itemsText(o)) + '</div>' +
      '<div class="meta">' + o.total + ' Tk, ordered at ' + timeText(o) + '</div>' +
      '<button class="small" data-act="advance" data-id="' + o.id + '">' + NEXT_LABEL[o.status] +
          '</button>' +
    '</div></div>';
}

function advanceOrder(id) {
  const o = orders.find(function (x) { return x.id === id; });
  if (!o) return;
  const next = STATUS_FLOW[STATUS_FLOW.indexOf(o.status) + 1];
  if (!next) return;
  updateDoc(doc(db, "orders", id), { status: next }).catch(function (err) {
    toast("Could not update the order.");
    console.error(err);
  });
}

// ---------- end of day ----------
$("eod-btn").addEventListener("click", endOfDay);

async function endOfDay() {
  let all;
  try {
    all = await getDocs(collection(db, "orders"));
  } catch (err) {
    toast("Could not read orders. Try again.");
    console.error(err);
    return;
  }

  if (all.empty) {
    toast("No orders to close today.");
    return;
  }

  const activeCount = all.docs.filter(function (d) { return d.data().status !== "Done"; }).length;
  let msg = "Save today's report and remove all " + all.size + " orders? Token numbers will restart from
      1.";
  if (activeCount > 0) {
    msg += "\n\n" + activeCount + " order(s) are still active and will also be removed.";
  }
  if (!confirm(msg)) return;
  if (!confirm("This cannot be undone. Are you sure?")) return;

  // Build the summary before anything is deleted
  let totalSales = 0;
  const itemCount = {};
  all.docs.forEach(function (d) {
    const o = d.data();
    totalSales += Number(o.total) || 0;
    (o.items || []).forEach(function (i) {
      itemCount[i.name] = (itemCount[i.name] || 0) + (Number(i.qty) || 0);
    });
  });

  const btn = $("eod-btn");
  btn.disabled = true;
  btn.textContent = "Closing the day...";

  try {
    // 1. Save the report first. If this fails, nothing is deleted.
```

```
    await addDoc(collection(db, "daily_reports"), {
      date: todayKey(),
      totalOrders: all.size,
      totalSales: totalSales,
      items: itemCount,
      activeAtClose: activeCount,
      createdAt: serverTimestamp()
    });

    // 2. Delete all orders in batches (Firestore allows up to 500 per batch)
    const docs = all.docs;
    for (let i = 0; i < docs.length; i += 400) {
      const batch = writeBatch(db);
      docs.slice(i, i + 400).forEach(function (d) { batch.delete(d.ref); });
      await batch.commit();
    }

    // 3. Restart token numbers from 1
    await setDoc(doc(db, "counters", "orders"), { last: 0 });

    toast("Day closed. Report saved and tokens restart from 1.");
  } catch (err) {
    toast("Could not close the day. Nothing was deleted if the report failed. Try again.");
    console.error(err);
  }

  btn.disabled = false;
  btn.textContent = "End of day: save report and reset tokens";
}

// ---------- past reports ----------
function watchReports() {
  onSnapshot(query(collection(db, "daily_reports"), orderBy("createdAt", "desc"), limit(7)),
    function (snap) {
      if (snap.empty) {
        $("reports-list").innerHTML = '<div class="empty">No reports yet. A report is saved each time
            you press End of day.</div>';
        return;
      }
      $("reports-list").innerHTML = snap.docs.map(function (d) {
        const r = d.data();
        const top = Object.keys(r.items || {})
          .map(function (name) { return { name: name, qty: r.items[name] }; })
          .sort(function (a, b) { return b.qty - a.qty; })
          .slice(0, 5)
          .map(function (i) { return i.name + " x" + i.qty; })
          .join(", ");
        return '<div class="list-row">' +
          '<span><b>' + escapeHtml(r.date || "") + '</b>: ' + (r.totalOrders || 0) + ' orders, ' +
          (r.totalSales || 0) + ' Tk<br><span class="muted">Top items: ' + escapeHtml(top || "none") +
              '</span></span>' +
        '</div>';
      }).join("");
    },
    function (err) { console.error(err); }
  );
}

// ---------- menu ----------
function watchMenu() {
  onSnapshot(query(collection(db, "menu"), orderBy("order")),
    function (snap) {
      menu = snap.docs.map(function (d) { return Object.assign({ id: d.id }, d.data()); });
      renderMenu();
      renderWaste();
    },
    function (err) { toast("Could not load the menu."); console.error(err); }
  );
}

function renderMenu() {
  const rows = menu.map(function (f) {
    const price = f.discount > 0
      ? '<s class="off-text">' + f.price + '</s> <b>' + finalPrice(f) + ' Tk</b> <span class="badge">' +
          f.discount + '% off</span>'
      : f.price + ' Tk';
    const eat = (f.minutes || DEFAULT_EAT_MIN) + ' min';
    return '<tr>' +
```

```
      '<td data-label="Item">' + escapeHtml(f.name) + '</td>' +
      '<td data-label="Price">' + price + '</td>' +
      '<td data-label="Eat time">' + eat + '</td>' +
      '<td data-label="Status"><span class="badge ' + (f.available ? "green" : "off") + '">' +
          (f.available ? "Available" : "Sold out") + '</span></td>' +
      '<td class="actions">' +
        '<button class="ghost small" data-act="toggle" data-id="' + f.id + '">' + (f.available ? "Mark
            sold out" : "Mark available") + '</button>' +
        '<button class="ghost small" data-act="edit" data-id="' + f.id + '">Edit</button>' +
        '<button class="danger small" data-act="delete" data-id="' + f.id + '">Delete</button>' +
      '</td></tr>';
  });
  $("menu-body").innerHTML = rows.length
    ? rows.join("")
    : '<tr><td colspan="5" class="empty">No items yet. Add one below.</td></tr>';
}

async function addFood() {
  const name = $("food-name").value.trim();
  const price = parseInt($("food-price").value, 10);
  const minutes = parseInt($("food-minutes").value, 10);
  if (!name || isNaN(price) || price < 0) {
    toast("Enter a food name and a valid price.");
    return;
  }
  try {
    await addDoc(collection(db, "menu"), {
      name: name,
      price: price,
      minutes: isNaN(minutes) || minutes < 1 ? DEFAULT_EAT_MIN : minutes,
      available: true,
      discount: 0,
      order: Date.now()
    });
    $("food-name").value = "";
    $("food-price").value = "";
    $("food-minutes").value = "";
    toast(name + " added to the menu.");
  } catch (err) {
    toast("Could not add the item.");
    console.error(err);
  }
}

function toggleAvailable(id) {
  const f = menu.find(function (x) { return x.id === id; });
  if (!f) return;
  updateDoc(doc(db, "menu", id), { available: !f.available });
}

function editFood(id) {
  const f = menu.find(function (x) { return x.id === id; });
  if (!f) return;
  const name = prompt("Food name:", f.name);
  if (name === null) return;
  const priceText = prompt("Price (Tk):", f.price);
  if (priceText === null) return;
  const minText = prompt("Eat time (minutes):", f.minutes || DEFAULT_EAT_MIN);
  if (minText === null) return;
  const price = parseInt(priceText, 10);
  const minutes = parseInt(minText, 10);
  if (!name.trim() || isNaN(price) || price < 0 || isNaN(minutes) || minutes < 1) {
    toast("Name, price or eat time is not valid.");
    return;
  }
  updateDoc(doc(db, "menu", id), { name: name.trim(), price: price, minutes: minutes });
  toast("Menu item updated.");
}

function deleteFood(id) {
  const f = menu.find(function (x) { return x.id === id; });
  if (!f) return;
  if (!confirm("Remove " + f.name + " from the menu?")) return;
  deleteDoc(doc(db, "menu", id));
}

$("add-food-btn").addEventListener("click", addFood);
```

```
// ---------- food waste saver ----------
function renderWaste() {
  const available = menu.filter(function (f) { return f.available; });
  $("waste-food").innerHTML = available.map(function (f) {
    return '<option value="' + f.id + '">' + escapeHtml(f.name) + ' (' + f.price + ' Tk)</option>';
  }).join("");

  const discounted = menu.filter(function (f) { return f.discount > 0; });
  $("waste-list").innerHTML = discounted.length
    ? discounted.map(function (f) {
        return '<div class="list-row">' +
          '<span><b>' + escapeHtml(f.name) + '</b>: ' + f.discount + '% off, now ' + finalPrice(f) + '
              Tk</span>' +
          '<button class="ghost small" data-act="stopDiscount" data-id="' + f.id + '">Stop
              discount</button>' +
        '</div>';
      }).join("")
    : '<div class="empty">No discounts running. Pick a food above to start one.</div>';
}

function setDiscount(id, percent) {
  updateDoc(doc(db, "menu", id), { discount: percent }).catch(function (err) {
    toast("Could not save the discount.");
    console.error(err);
  });
}

$("waste-start").addEventListener("click", function () {
  const id = $("waste-food").value;
  const percent = parseInt($("waste-percent").value, 10);
  if (!id) { toast("Choose a food item first."); return; }
  if (isNaN(percent) || percent < 5 || percent > 90) {
    toast("Discount must be between 5 and 90.");
    return;
  }
  setDiscount(id, percent);
  toast("Discount started.");
});

// ---------- planner settings ----------
const PLANNER_DEFAULTS = { minBreak: 10, maxBreak: 45, step: 5, defaultEat: 5 };

async function loadPlannerSettings() {
  try {
    const snap = await getDoc(doc(db, "settings", "planner"));
    const s = Object.assign({}, PLANNER_DEFAULTS, snap.exists() ? snap.data() : {});
    $("pl-min").value = s.minBreak;
    $("pl-max").value = s.maxBreak;
    $("pl-step").value = s.step;
    $("pl-default").value = s.defaultEat;
  } catch (err) {
    console.error(err);
  }
}

$("planner-save").addEventListener("click", async function () {
  const minBreak = parseInt($("pl-min").value, 10);
  const maxBreak = parseInt($("pl-max").value, 10);
  const step = parseInt($("pl-step").value, 10);
  const defaultEat = parseInt($("pl-default").value, 10);
  if (isNaN(minBreak) || isNaN(maxBreak) || minBreak < 1 || maxBreak <= minBreak) {
    toast("Longest break must be bigger than shortest break.");
    return;
  }
  if (isNaN(step) || step < 1 || isNaN(defaultEat) || defaultEat < 1) {
    toast("Step and default eat time must be at least 1.");
    return;
  }
  try {
    await setDoc(doc(db, "settings", "planner"), {
      minBreak: minBreak, maxBreak: maxBreak, step: step, defaultEat: defaultEat
    });
    toast("Planner settings saved.");
  } catch (err) {
    toast("Could not save planner settings.");
    console.error(err);
  }
});
```

```
</script>
</body>
</html>
```

B.6 style.css

```
/* ========================================================
   BreakBite shared stylesheet: "Spice Kitchen" theme
   Change only the colours in :root to re-theme the whole site.
   Food doodles and animations are included; no HTML change needed.
   ======================================================== */

:root {
  /* ---- Main colours ---- */
  --bg: #fff3dc;          /* page background: warm cream */
  --ink: #3b2314;         /* text and borders: cocoa brown */
  --accent: #d9482b;      /* hover, highlight: chilli red */
  --card: #ffffff;        /* cards and header */
  --danger: #b02a1a;      /* clear / remove buttons */
  --leaf: #4c8c4a;        /* fresh green */
  --turmeric: #f2b33d;    /* warm yellow highlight */

  /* ---- Ticket and status colours ---- */
  --t-yellow: #ffe08a;    /* Received */
  --t-blue: #ffd0a8;      /* Preparing (peach) */
  --t-mint: #cfeab0;      /* Ready (fresh leaf) */

  /* ---- Soft tints ---- */
  --bg-soft: #fff8ec;
  --ink-soft: #7a5a44;
  --line: #f0d7b8;
  --column: rgba(255, 255, 255, .6);

  /* ---- Shape and shadow ---- */
  --radius: 16px;
  --shadow: 4px 4px 0 var(--ink);
  --shadow-small: 2px 2px 0 var(--ink);

  padding-top: env(safe-area-inset-top, 0px);
  padding-bottom: env(safe-area-inset-bottom, 0px);
}

* { box-sizing: border-box; }

/* Cream background with a faint food doodle pattern (burger, pizza, tea, samosa, fries, drumstick) */
body {
  margin: 0;
  background-color: var(--bg);
  background-image: url("data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg' width='320'
      height='320' viewBox='0 0 320 320'><g fill='none' stroke='%233b2314' stroke-width='3'
      stroke-linecap='round' stroke-linejoin='round' opacity='0.10'><path d='M18 60 Q40 28 62 60
      Z'/><path d='M16 66 H64'/><path d='M18 72 Q40 80 62 72'/><path d='M16 78 H64'/><path d='M18 84 Q40
      93 62 84 Z'/><path d='M160 40 L192 104 L128 104 Z'/><circle cx='158' cy='72' r='3'/><circle
      cx='170' cy='86' r='3'/><circle cx='150' cy='92' r='3'/><path d='M238 62 H272 V82 Q272 98 255 98
      Q238 98 238 82 Z'/><path d='M272 68 Q284 68 284 78 Q284 86 272 86'/><path d='M247 52 Q244 45 248
      38'/><path d='M258 52 Q255 45 259 38'/><path d='M36 178 L68 146 L100 178 Z'/><path d='M52 168 L84
      168'/><path d='M146 192 L152 160 H184 L190 192 Z'/><path d='M158 160 V148'/><path d='M168 160
      V146'/><path d='M178 160 V149'/><path d='M244 206 Q234 196 242 184 Q254 176 266 182 Q280 188 272
      204 Q266 216 244 206 Z'/><path d='M266 180 L280 168'/><path d='M120 270 Q112 258 122 250 Q136 246
      146 258 Q148 270 136 274 Q126 276 120 270 Z'/><path d='M230 280 Q242 262 262 268 Q278 276 268 292
      Q254 302 230 280 Z'/></g></svg>");
  background-size: 320px 320px;
  color: var(--ink);
  font-family: "Trebuchet MS", "Segoe UI", system-ui, sans-serif;
  line-height: 1.5;
  overflow-x: hidden;
  animation: fadeIn .6s ease both;
}

h1, h2, h3 { margin: 0; line-height: 1.2; letter-spacing: -.3px; }
.muted { color: var(--ink-soft); }
.hidden { display: none !important; }

/* ---------- Buttons ---------- */
button {
  font: inherit;
  border: 2px solid var(--ink);
  border-radius: 12px;
  padding: 9px 16px;
  background: var(--accent);
  color: #ffffff;
  font-weight: 700;
  cursor: pointer;
  box-shadow: var(--shadow-small);
```

```
  transition: transform .15s, box-shadow .15s, background .15s, color .15s;
  white-space: nowrap;
}
button:hover {
  background: var(--ink);
  color: #ffffff;
  transform: translate(-2px, -2px);
  box-shadow: 4px 4px 0 var(--accent);
  animation: wiggle .5s ease;
}
button:active { transform: translate(2px, 2px); box-shadow: none; }
button:disabled {
  opacity: .5;
  cursor: not-allowed;
  transform: none;
  box-shadow: var(--shadow-small);
  background: var(--ink-soft);
  animation: none;
}
button.ghost { background: var(--card); color: var(--ink); }
button.ghost:hover { background: var(--bg-soft); color: var(--ink); box-shadow: 4px 4px 0
    var(--turmeric); }
button.small { padding: 5px 11px; font-size: 14px; }
button.danger { background: var(--card); color: var(--danger); border-color: var(--danger); }
button.danger:hover { background: var(--danger); color: #ffffff; box-shadow: 4px 4px 0 var(--ink); }
button:focus-visible, input:focus-visible, select:focus-visible, a:focus-visible {
  outline: 3px solid var(--turmeric);
  outline-offset: 2px;
}

/* Spinner: shown on any button that has the class "loading" */
button.loading { pointer-events: none; }
button.loading::after {
  content: "";
  display: inline-block;
  width: 14px; height: 14px;
  margin-left: 8px;
  vertical-align: -2px;
  border: 3px solid #ffffff;
  border-right-color: transparent;
  border-radius: 50%;
  animation: spin .7s linear infinite;
}

/* ---------- Inputs ---------- */
input, select {
  font: inherit;
  padding: 9px 12px;
  border: 2px solid var(--ink);
  border-radius: 12px;
  background: var(--card);
  color: var(--ink);
  max-width: 100%;
}
label { display: block; font-weight: 700; font-size: 14px; }
label.field { margin-bottom: 12px; }
label input, label.field input { display: block; width: 100%; margin-top: 4px; }

/* ---------- Header and tabs ---------- */
header.top {
  background: var(--card);
  border-bottom: 3px solid var(--ink);
  padding: 12px 24px;
  display: flex;
  align-items: center;
  gap: 16px;
  flex-wrap: wrap;
  position: sticky;
  top: 0;
  z-index: 5;
}
.brand { font-size: 24px; font-weight: 900; letter-spacing: -1px; color: var(--ink); }
.logo {
  display: inline-grid; place-items: center;
  width: 32px; height: 32px; margin-right: 8px; vertical-align: -7px;
  border-radius: 9px;
  background: var(--accent); color: var(--card);
  font-size: 19px; font-weight: 900; letter-spacing: 0;
```

```
  box-shadow: var(--shadow-small);
}
.tabs { display: flex; gap: 6px; flex-wrap: wrap; margin-left: 16px; }
.tab {
  background: transparent;
  color: var(--ink);
  border: 2px solid transparent;
  box-shadow: none;
  text-decoration: none;
  padding: 8px 14px;
  border-radius: 999px;
}
.tab:hover { background: var(--bg-soft); color: var(--accent); transform: none; box-shadow: none;
    animation: none; }
.tab.active {
  background: var(--turmeric);
  border-color: var(--ink);
  box-shadow: var(--shadow-small);
  color: var(--ink);
}
header.top .right { margin-left: auto; }

/* ---------- Layout ---------- */
main { max-width: 760px; margin: 0 auto; padding: 24px; }
main.wide { max-width: 1100px; }
main.narrow { max-width: 760px; margin: 0 auto; padding: 24px; }
.panel { display: none; }
.panel.active { display: block; animation: reveal .45s ease both; }
.view { display: none; }
.view.active { display: block; animation: reveal .45s ease both; }

/* ---------- Cards ---------- */
.card {
  background: var(--card);
  border: 2px solid var(--ink);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 20px;
  margin-bottom: 20px;
}
.card h2, .card h3 { font-size: 18px; margin-bottom: 12px; }
a.card { color: var(--ink); display: block; text-decoration: none; transition: transform .15s,
    box-shadow .15s; }
a.card:hover { transform: translate(-2px, -2px); box-shadow: 6px 6px 0 var(--accent); }
a.card h2 { color: var(--accent); margin-bottom: 6px; }
a.card p { margin: 0; color: var(--ink-soft); }
a.card .go { display: inline-block; margin-top: 12px; font-weight: 800; color: var(--ink); }
.grid { display: grid; grid-template-columns: repeat(2, 1fr); gap: 16px; }
.soon {
  margin-top: 16px;
  background: var(--turmeric);
  border: 2px dashed var(--ink);
  border-radius: var(--radius);
  padding: 18px 22px;
  color: var(--ink);
  font-weight: 600;
}

/* ---------- Hero (top banner) ---------- */
.hero { margin-bottom: 20px; }
.hero h1 { font-size: 30px; margin-bottom: 6px; color: var(--accent); }
.hero p { margin: 0; color: var(--ink-soft); }

/* ---------- Rush radar ---------- */
.rush { display: flex; justify-content: space-between; align-items: center; gap: 12px; }
.rush b { font-weight: 800; }
.rush .num, .rush #rush-num { font-size: 38px; font-weight: 900; color: var(--accent); }

/* ---------- Menu items: lift slightly on hover ---------- */
.item {
  display: flex; justify-content: space-between; align-items: center; gap: 12px;
  padding: 12px 10px;
  margin: 0 -10px;
  border-bottom: 2px dashed var(--line);
  border-radius: 12px;
  transition: transform .2s ease, background .2s ease, box-shadow .2s ease;
}
.item:last-child { border-bottom: 0; }
```

```
.item:hover {
  transform: translate(-2px, -2px);
  background: var(--bg-soft);
  box-shadow: 3px 3px 0 var(--turmeric);
}
.item > div:first-child { min-width: 0; flex: 1; }
.name { font-weight: 700; font-size: 16px; overflow-wrap: anywhere; }
.item .meta { color: var(--ink-soft); font-size: 14px; }
.badge {
  display: inline-block;
  padding: 2px 10px;
  border: 2px solid var(--ink);
  border-radius: 999px;
  font-size: 13px;
  font-weight: 700;
  background: var(--turmeric);
  color: var(--ink);
  white-space: nowrap;
}
.badge.green { background: var(--t-mint); }
.badge.off { background: var(--bg-soft); color: var(--ink-soft); }
.stepper { display: flex; align-items: center; gap: 10px; flex-shrink: 0; }
.stepper span {
  min-width: 24px; text-align: center; font-weight: 800;
  transition: transform .2s ease;
}
.price { font-weight: 900; white-space: nowrap; color: var(--accent); }
.off-text { color: var(--ink-soft); }
.empty { color: var(--ink-soft); padding: 8px 0; }

/* ---------- List rows (Running discounts, Done orders) ---------- */
.list-row {
  display: flex; justify-content: space-between; align-items: center;
  gap: 12px; flex-wrap: wrap;
  padding: 10px 0;
  border-bottom: 2px dashed var(--line);
}
.list-row:last-child { border-bottom: 0; }
.list-row > span { min-width: 0; overflow-wrap: anywhere; flex: 1 1 200px; }
.list-row button { flex-shrink: 0; }

/* ---------- Form row (inputs and button in one row) ---------- */
.form-row { display: flex; gap: 10px; flex-wrap: wrap; align-items: center; }
.form-row input, .form-row select { flex: 1 1 160px; min-width: 0; }
.form-row button { flex-shrink: 0; }

/* ---------- Cart bar (student): slides up when the page opens ---------- */
.cart {
  position: sticky; bottom: 12px;
  background: var(--ink); color: var(--card);
  border-radius: var(--radius);
  padding: 14px 18px;
  margin-bottom: 20px;
  display: flex; justify-content: space-between; align-items: center; gap: 12px;
  box-shadow: 4px 4px 0 var(--accent);
  animation: slideUp .6s cubic-bezier(.2, .9, .3, 1.2) both;
}
.cart button { background: var(--turmeric); color: var(--ink); border-color: var(--turmeric); }
.cart button:hover { background: var(--accent); color: #ffffff; border-color: var(--accent); }

/* ---------- Token tracking (student) ---------- */
.ticket-big { text-align: center; padding: 8px 0; }
.ticket-big small { color: var(--ink-soft); font-size: 14px; }
.ticket-big .tok {
  font-size: 56px; font-weight: 900; line-height: 1.1;
  color: var(--accent);
  display: inline-block;
}
/* Token pulses while the order is Ready (the third step is the current one) */
.card:has(.step:nth-child(3).now) .tok { animation: pulse 1.1s ease-in-out infinite; }

.steps { display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px; margin: 16px 0; }
.step {
  text-align: center; padding: 12px 6px; border-radius: 12px;
  border: 2px solid var(--ink);
  background: var(--card); color: var(--ink-soft);
  font-weight: 700; font-size: 14px;
  transition: background .4s ease, color .4s ease;
```

```
}
.step span { display: block; font-size: 20px; font-weight: 900; }
.step.done { background: var(--t-mint); color: var(--ink); }
/* The current step glows softly */
.step.now {
  background: var(--accent); color: #ffffff;
  box-shadow: var(--shadow-small);
  animation: glow 1.4s ease-in-out infinite;
}
.track-msg { text-align: center; font-weight: 700; margin-bottom: 12px; }

/* Token card fades in when it appears */
#track-card:not([hidden]) { animation: reveal .5s ease both; }

/* ---------- Forms and planner ---------- */
.form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }
.summary {
  display: flex; justify-content: space-between; gap: 12px; flex-wrap: wrap;
  background: var(--t-yellow);
  border: 2px solid var(--ink);
  border-radius: 12px;
  padding: 12px 16px;
  margin-bottom: 12px;
}
.summary b { font-size: 18px; }
.error { color: var(--danger); margin: 8px 0 0; min-height: 22px; font-weight: 700; }

/* ---------- Staff: stats ---------- */
.stats { display: grid; grid-template-columns: repeat(3, 1fr); gap: 14px; margin-bottom: 14px; }
.stat {
  background: var(--card);
  border: 2px solid var(--ink);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 16px;
}
.stat .muted { font-size: 15px; }
.stat .num { font-size: 34px; font-weight: 900; color: var(--ink); }
.stat.hot .num { color: var(--accent); }
.staff-actions { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 16px; }

/* ---------- Staff: order board ---------- */
.board { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; align-items: start;
    margin-bottom: 20px; }
.col {
  background: var(--column);
  border: 2px dashed var(--ink);
  border-radius: var(--radius);
  padding: 12px;
  min-height: 150px;
}
.col h3 { font-size: 16px; margin-bottom: 10px; display: flex; justify-content: space-between;
    align-items: center; }
.count {
  background: var(--accent); color: var(--card);
  border-radius: 999px; padding: 0 10px; font-size: 14px; font-weight: 700;
}

.ticket {
  background: var(--card);
  border: 2px solid var(--ink);
  border-radius: 12px;
  margin-bottom: 10px;
  display: flex;
  overflow: hidden;
  box-shadow: var(--shadow-small);
}
.stub {
  width: 78px; flex-shrink: 0;
  display: flex; flex-direction: column;
  align-items: center; justify-content: center;
  padding: 10px 4px;
  border-right: 2px dashed var(--ink);
}
.stub small { font-size: 12px; color: var(--ink-soft); }
.stub b { font-size: 26px; font-weight: 900; color: var(--ink); }

/* Status colours: Received = turmeric, Preparing = peach, Ready = fresh leaf */
```

```
.ticket.Received .stub, .ticket.Pending .stub, .ticket.t-r .stub { background: var(--t-yellow); }
.ticket.Preparing .stub, .ticket.t-p .stub { background: var(--t-blue); }
.ticket.Ready .stub, .ticket.t-d .stub { background: var(--t-mint); }

.tb, .ticket .body { padding: 10px 12px; flex: 1; min-width: 0; }
.tb .it, .ticket .items { font-weight: 700; word-break: break-word; }
.tb .mt, .ticket .meta { font-size: 13px; color: var(--ink-soft); margin: 4px 0 8px; }
.ticket button { margin-top: 2px; }

/* ---------- Staff: menu table ---------- */
.table-wrap { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; }
th, td { text-align: left; padding: 10px 8px; border-bottom: 2px dashed var(--line); vertical-align:
    middle; }
th { font-size: 14px; color: var(--ink-soft); font-weight: 700; }
td.actions { display: flex; flex-wrap: wrap; gap: 6px; }

/* ---------- Login (staff) ---------- */
#login-screen { min-height: 100vh; display: grid; grid-template-columns: 1fr 1fr; }
.login-brand {
  background: var(--accent);
  color: var(--card);
  padding: 48px;
  display: flex; flex-direction: column; justify-content: center; gap: 16px;
  border-right: 3px solid var(--ink);
}
.login-brand .logo {
  display: inline-block;
  width: auto; height: auto;
  padding: 10px 20px;
  font-size: 48px; font-weight: 900; line-height: 1.1; letter-spacing: -1px;
  background: var(--turmeric);
  color: var(--ink);
  border-radius: 14px;
  box-shadow: 4px 4px 0 var(--ink);
  margin: 0;
}
.login-brand p { margin: 0; font-size: 18px; max-width: 32ch; color: var(--card); }
.login-form { display: flex; align-items: center; justify-content: center; padding: 32px; background:
    var(--bg); }
.login-form form { width: 100%; max-width: 340px; }
.login-form h2 { font-size: 26px; margin-bottom: 4px; color: var(--accent); }
.login-form .msg { min-height: 24px; color: var(--danger); margin: 8px 0; font-weight: 700; }
.login-form button { width: 100%; padding: 12px; }

/* ---------- Toast ---------- */
#toast {
  position: fixed; bottom: 80px; left: 50%; transform: translateX(-50%);
  background: var(--ink); color: var(--card);
  padding: 10px 18px; border: 2px solid var(--ink);
  border-radius: 12px; z-index: 10; font-weight: 700;
  max-width: calc(100vw - 32px); text-align: center;
  animation: reveal .3s ease both;
}

/* ---------- Keyframes ---------- */
@keyframes wiggle {
  0%, 100% { transform: translate(-2px, -2px) rotate(0); }
  25% { transform: translate(-2px, -2px) rotate(-2deg); }
  75% { transform: translate(-2px, -2px) rotate(2deg); }
}
@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.12); }
}
@keyframes glow {
  0%, 100% { box-shadow: 0 0 0 0 rgba(217, 72, 43, .45); }
  50% { box-shadow: 0 0 0 7px rgba(217, 72, 43, 0); }
}
@keyframes spin { to { transform: rotate(360deg); } }
@keyframes reveal {
  from { opacity: 0; transform: translateY(12px); }
  to { opacity: 1; transform: none; }
}
@keyframes slideUp {
  from { opacity: 0; transform: translateY(140%); }
  to { opacity: 1; transform: translateY(0); }
}
```

```
@keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }

/* ---------- Responsive (phones) ---------- */
@media (max-width: 700px) {
  main, main.narrow { padding: 16px; }
  header.top { padding: 12px 16px; gap: 10px; }
  .tabs {
    margin-left: 0; width: 100%;
    flex-wrap: nowrap; overflow-x: auto;
    -webkit-overflow-scrolling: touch;
    padding-bottom: 4px;
  }
  .tab { flex-shrink: 0; white-space: nowrap; }

  .form-grid, .stats, .board, .grid { grid-template-columns: 1fr; }
  .hero h1 { font-size: 24px; }
  .steps { gap: 6px; }

  .item { flex-wrap: wrap; }
  .item > div:first-child { flex: 1 1 60%; }
  .stepper { margin-left: auto; }

  .stub { width: 66px; }
  .stub b { font-size: 22px; }
  .tb, .ticket .body { padding: 10px; }

  .list-row { flex-direction: column; align-items: flex-start; }
  .list-row button { width: auto; }

  .form-row input, .form-row select, .form-row button { flex: 1 1 100%; width: 100%; }

  .table-wrap { overflow-x: visible; }
  table, thead, tbody, th, td, tr { display: block; width: 100%; }
  thead { display: none; }
  tr {
    border: 2px solid var(--ink);
    border-radius: 12px;
    padding: 10px;
    margin-bottom: 12px;
    background: var(--card);
    box-shadow: var(--shadow-small);
  }
  td { border: none; padding: 4px 0; }
  td.actions { margin-top: 6px; }

  #login-screen { grid-template-columns: 1fr; }
  .login-brand { padding: 32px 24px; }
}

/* Users who prefer less motion get no animation */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation: none !important; transition: none !important; }
}
```

B.7 README.md

```
# BreakBite

A smart canteen web project for the Gono Bishwabidyalay CSE Software Development Contest 2026 (Category
    1: Semester 1st to 4th).

## About

BreakBite helps students spend less time waiting in the university canteen. Students can see the menu,
    order ahead, get a token and know the waiting time. Staff can manage orders and food from their own
    page.

## Features

- Digital Menu
- Token System
- Waiting Time
- Pre-order
- Rush Radar
- Break Planner
- Green Token
- Food Waste Saver

## Team

| Name | Student ID | Responsibility |
|---|---|---|
| Nahar Mim (Team Lead) | 26251037 | Lead and documentation |
| Sadnan Ahmed | 26251038 | Student pages |
| Wasika Jahan | 26251026 | UI design |
| Mirza Muin Labbaik | 26251021 | Staff page and database |
| Habiba Jannat Taiyaba | 26251011 | Break Planner and testing |

## Folder Structure

- `/index.html` : home page (Mim)
- `/student.html` : student pages (Sadnan)
- `/staff.html` : staff page and database (Labbaik)
- `/planner.html` : Break Planner and testing (Taiyaba)
- `/style.css` : UI design and styling (Wasika)
- `README.md` : project information (Mim)

## How to Work Together

1. Open GitHub Desktop and click Fetch origin, then Pull origin before starting work.
2. Work only on your own files so changes do not clash.
3. Click Commit with a short message about what you changed.
4. Click Push origin.

## Important Dates

- Final project submission: 12 November 2026
- Showcase and poster: 16 November 2026
```

B.8 .gitattributes

```
# Auto detect text files and perform LF normalization
* text=auto
```

B.9 index.html (new splash page)

Replaces the earlier index.html. The Bite button opens home.html.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BreakBite</title>
  <link rel="stylesheet" href="style.css">
  <link rel="stylesheet" href="logo.css">
  <style>
    .splash-main {
      min-height: 100vh;
      display: flex;
      align-items: center;
      justify-content: center;
      padding: 24px;
    }
  </style>
</head>
<body>

<main class="splash-main">
  <div class="bb-logo-wrap">
    <div class="bb-logo">
      <svg class="bb-cookie" id="bbCookie" viewBox="0 0 220 220" aria-hidden="true">
        <defs>
          <mask id="bbBiteMask">
            <rect width="220" height="220" fill="#fff"/>
            <circle class="bb-bite-cut" cx="176" cy="60" r="34" fill="#000"/>
          </mask>
        </defs>
        <circle cx="116" cy="118" r="88" fill="#3b2314"/>
        <g mask="url(#bbBiteMask)">
          <circle cx="106" cy="108" r="88" fill="#f2b33d" stroke="#3b2314" stroke-width="7"/>
          <circle cx="70" cy="90" r="8" fill="#3b2314"/>
          <circle cx="112" cy="70" r="8" fill="#3b2314"/>
          <circle cx="96" cy="140" r="8" fill="#3b2314"/>
          <circle cx="140" cy="122" r="8" fill="#3b2314"/>
          <circle cx="66" cy="150" r="8" fill="#3b2314"/>
        </g>
        <circle class="bb-bite-cut" cx="176" cy="60" r="34" fill="none" stroke="#3b2314"
         stroke-width="7"/>
        <g class="bb-crumbs">
          <rect x="150" y="18" width="24" height="24" rx="6" fill="#d9482b" stroke="#3b2314"
           stroke-width="5"/>
          <rect x="182" y="18" width="24" height="24" rx="6" fill="#cfeab0" stroke="#3b2314"
           stroke-width="5"/>
          <rect x="182" y="50" width="24" height="24" rx="6" fill="#f2b33d" stroke="#3b2314"
           stroke-width="5"/>
        </g>
      </svg>
      <div class="bb-word">
        <span class="bb-break">Break</span>
        <button class="bb-bite" id="bbBite" type="button" aria-label="Bite: enter the site">Bite</button>
      </div>
    </div>
    <div class="bb-hint" id="bbHint">Click <b>Bite</b> to order food <span
     class="bb-arrow">&uarr;</span></div>
  </div>
</main>

<script>
  document.getElementById('bbBite').addEventListener('click', function () {
    window.location.href = 'home.html';
  });
</script>
</body>
</html>
```

B.10 logo.css

Styles and animations of the logo. All class names start with bb-.

```
/* BreakBite animated logo */
.bb-logo-wrap{display:flex;flex-direction:column;align-items:center;text-align:center;margin:0 0 28px}
.bb-logo{display:flex;align-items:center;gap:clamp(10px,2.4vw,22px);flex-wrap:wrap;justify-content:center}

.bb-cookie{width:clamp(96px,14vw,150px);height:auto;overflow:visible;animation:bb-float 3.2s ease-in-out
 infinite;transform-origin:50% 60%}
.bb-cookie .bb-crumbs{transform-box:fill-box;transform-origin:center;animation:bb-twinkle 2.4s ease-in-out
  infinite}
.bb-cookie .bb-bite-cut{transform-box:fill-box;transform-origin:center;transition:transform .5s ease}
.bb-cookie.bb-chomp{animation:bb-chomp .7s cubic-bezier(.3,.7,.3,1) both}
.bb-cookie.bb-chomp .bb-bite-cut{transform:translate(10px,-6px) scale(1.25)}

.bb-word{display:flex;align-items:center;gap:10px;flex-wrap:wrap;justify-content:center}
.bb-break{font-size:clamp(38px,6vw,64px);font-weight:900;letter-spacing:-2px;color:var(--ink,#3b2314);anim
 ation:bb-slide .8s .1s cubic-bezier(.2,.9,.3,1.2) both}

.bb-bite{position:relative;font:inherit;font-size:clamp(38px,6vw,64px);font-weight:900;letter-spacing:-2px
 ;color:#fff5e6;background:var(--accent,#d9482b);border:4px solid
 var(--ink,#3b2314);border-radius:20px;padding:2px 18px 8px;box-shadow:7px 7px 0
 var(--ink,#3b2314);cursor:pointer;animation:bb-slide .8s .25s cubic-bezier(.2,.9,.3,1.2) both,bb-bob 2.6s
  1.2s ease-in-out infinite;transition:transform .15s,box-shadow .15s,background .15s}
.bb-bite:hover{background:#c23b1f;transform:translate(-3px,-3px) rotate(-2deg);box-shadow:10px 10px 0
 var(--ink,#3b2314);animation-play-state:paused}
.bb-bite:active{transform:translate(4px,4px);box-shadow:3px 3px 0 var(--ink,#3b2314)}
.bb-bite:focus-visible{outline:4px solid var(--gold,#f2b33d);outline-offset:6px}
.bb-bite.bb-pressed{animation:bb-squash .45s ease both}

.bb-hint{margin-top:22px;display:inline-flex;align-items:center;gap:10px;background:var(--gold,#f2b33d);bo
 rder:3px solid var(--ink,#3b2314);border-radius:999px;padding:9px
 18px;font-weight:800;font-size:clamp(14px,2vw,17px);color:var(--ink,#3b2314);box-shadow:4px 4px 0
 var(--ink,#3b2314);animation:bb-hintPulse 1.8s ease-in-out infinite}
.bb-hint .bb-arrow{display:inline-block;animation:bb-up 1s ease-in-out infinite}
.bb-hint.bb-hide{opacity:0;transition:opacity .3s}

.bb-crumb{position:fixed;width:12px;height:12px;border-radius:50%;background:var(--gold,#f2b33d);border:2p
 x solid var(--ink,#3b2314);pointer-events:none;animation:bb-fly .8s ease-out forwards;z-index:50}

@keyframes bb-float{0%,100%{transform:translateY(0) rotate(0)}50%{transform:translateY(-8px)
 rotate(3deg)}}
@keyframes bb-twinkle{0%,100%{transform:scale(1) rotate(0)}50%{transform:scale(1.15) rotate(8deg)}}
@keyframes bb-slide{from{opacity:0;transform:translateY(24px) scale(.9)}to{opacity:1;transform:none}}
@keyframes bb-bob{0%,100%{transform:translateY(0)}50%{transform:translateY(-6px)}}
@keyframes bb-chomp{0%{transform:scale(1) rotate(0)}30%{transform:scale(1.12,.9)
 rotate(-6deg)}60%{transform:scale(.96,1.04) rotate(3deg)}100%{transform:scale(1) rotate(0)}}
@keyframes bb-squash{0%{transform:scale(1)}35%{transform:scale(1.12,.86)}70%{transform:scale(.94,1.06)}100
 %{transform:scale(1)}}
@keyframes bb-hintPulse{0%,100%{box-shadow:4px 4px 0 var(--ink,#3b2314)}50%{box-shadow:6px 6px 0
 var(--ink,#3b2314);transform:translateY(-2px)}}
@keyframes bb-up{0%,100%{transform:translateY(0)}50%{transform:translateY(-4px)}}
@keyframes bb-fly{0%{opacity:1;transform:translate(0,0)
 scale(1)}100%{opacity:0;transform:translate(var(--dx),var(--dy)) scale(.4) rotate(200deg)}}

@media (prefers-reduced-motion:reduce){.bb-logo-wrap
 *,.bb-crumb{animation:none!important;transition:none!important}}
```

B.11 logo.js

Chomp animation and crumbs, then opens the order page. Not linked by index.html yet (note 1).

```
/* BreakBite logo: clicking "Bite" chomps the cookie, then opens the order page */
(function () {
  var bite = document.getElementById("bbBite");
  var cookie = document.getElementById("bbCookie");
  var hint = document.getElementById("bbHint");
  if (!bite || !cookie) return;
  var TARGET = "student.html";
  var busy = false;

  function crumbs(x, y) {
    for (var i = 0; i < 9; i++) {
      var c = document.createElement("span");
      c.className = "bb-crumb";
      var angle = (Math.PI * 2 / 9) * i + Math.random() * 0.4;
      var dist = 60 + Math.random() * 60;
      c.style.left = x + "px";
      c.style.top = y + "px";
      c.style.setProperty("--dx", Math.cos(angle) * dist + "px");
      c.style.setProperty("--dy", Math.sin(angle) * dist + "px");
      document.body.appendChild(c);
      setTimeout(function (el) { el.remove(); }.bind(null, c), 900);
    }
  }

  bite.addEventListener("click", function () {
    if (busy) return;
    busy = true;
    if (hint) hint.classList.add("bb-hide");
    bite.classList.add("bb-pressed");
    cookie.classList.add("bb-chomp");
    var r = bite.getBoundingClientRect();
    crumbs(r.left + r.width / 2, r.top + r.height / 2);
    setTimeout(function () { window.location.href = TARGET; }, 800);
  });
})();
```